

# Procedural Terrain Generation with Real Time Erosion Simulation in PyGandalf Framework

Bachelor Thesis



**Author:** Marino Chiarugi (Student ID: 4653)

**Supervisor:** Prof. George Papagiannakis

**Co-Supervisor:** Dr. Emmanouil Kamarianakis

Department of Computer Science  
School of Sciences & Engineering  
University of Crete  
Heraklion, Greece

October 3, 2026

# Abstract

This thesis presents a real-time terrain generation and erosion simulation system developed in the PyGandalf Framework. The tool allows users to generate terrains while incorporating physically-based erosion processes on real-world heightmaps and featuring real-time water rendering. By leveraging compute shaders in the OpenGL API and parallel processing, the system ensures interactive performance suitable for real-time applications.

The key aspects of this system include the utilization of PyGandalf Framework as the foundational platform, the integration of a hydraulic erosion algorithm in compute shaders and the use of tessellation shaders in the rendering process of the terrain. Additionally, a user interface(UI) is provided that gives the option to change the parameters of the terrain generation and erosion simulation allowing for the creation of different types of environments. Furthermore, it encompasses a fragment shader that implements a physically based rendering pipeline and automatically shades the terrain using 4 textures based on height and slope.

In conclusion, this research contributes to the advancement of procedural terrain modeling by integrating erosion simulation and water rendering into a user-friendly, high-performance system that produces evolving landscapes that mimic natural terrain formation processes.

# Declaration of Authorship

I, Marino Chiarugi, declare that this thesis titled, "Procedural Terrain Generation with Real Time Erosion Simulation in PyGandalf Framework", and the work presented in it are my own. I confirm that:

- This work was done wholly or mainly while in candidature for a research degree at this University.
- Where any part of this thesis has previously been submitted for a degree or any other qualification at this University or any other institution, this has been clearly stated.
- Where I have consulted the published work of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this thesis is entirely my own work.
- I have acknowledged all main sources of help.
- Where the thesis is based on work done by myself jointly with others, I have made clear exactly what was done by others and what I have contributed myself.

**Signature:** Marino Chiarugi

---

**Date:** October 3, 2026

---

# Acknowledgements

I would like to express my gratitude to my supervising professor George Papiannakis for his continuous support and essential direction throughout the course of this project.

I would also like to thank Emmanouil Kamarianakis and Stratos Geronikolakis for their instructions and helpful advice.

I also extend my thanks to Karl from the ThinMatrix channel on YouTube for his helpful tutorials on water rendering, which greatly contributed to my understanding of the subject.

Lastly, I am grateful to Joey de Vries from LearnOpenGL.com for the tutorials and examples of basic graphics techniques implemented in OpenGL, as well as for his guidance in helping me understand how they work.

# Contents

|          |                                               |           |
|----------|-----------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                           | <b>1</b>  |
| 1.1      | Procedural generation . . . . .               | 1         |
| 1.1.1    | The perlin noise technique . . . . .          | 2         |
| 1.1.2    | Diffusion-Limited Aggregation (DLA) . . . . . | 3         |
| 1.1.3    | Marching cubes . . . . .                      | 3         |
| 1.2      | Erosion . . . . .                             | 4         |
| 1.3      | The pyGANDALF Framework . . . . .             | 6         |
| 1.4      | The Entity-Component-System . . . . .         | 7         |
| 1.5      | Thesis Overview . . . . .                     | 7         |
| <b>2</b> | <b>State of the Art</b>                       | <b>8</b>  |
| <b>3</b> | <b>Procedural Terrain Generation</b>          | <b>9</b>  |
| 3.1      | Heightmap . . . . .                           | 10        |
| 3.2      | Tessellation . . . . .                        | 13        |
| <b>4</b> | <b>Shading</b>                                | <b>16</b> |
| 4.1      | Physically Based Rendering . . . . .          | 17        |
| 4.2      | Height and Slope Blending . . . . .           | 19        |
| 4.2.1    | Blending . . . . .                            | 20        |
| 4.2.2    | Texture Bombing . . . . .                     | 22        |
| <b>5</b> | <b>Erosion Simulation</b>                     | <b>22</b> |
| <b>6</b> | <b>Implementation</b>                         | <b>27</b> |
| 6.1      | Terrain Generation System . . . . .           | 27        |
| 6.2      | Erosion System . . . . .                      | 27        |
| 6.3      | Water Rendering . . . . .                     | 30        |
| 6.4      | Performance . . . . .                         | 34        |
| <b>7</b> | <b>Conclusion</b>                             | <b>36</b> |
|          | <b>References</b>                             | <b>37</b> |

## List of Figures

|    |                                                                                                                                                                                                                                                                                                                                    |    |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1  | Minecraft, one of the most popular video games, uses procedural generation. Image taken from Minecraft website [22] . . . . .                                                                                                                                                                                                      | 1  |
| 2  | Showcase of the algorithm described in [3]. . . . .                                                                                                                                                                                                                                                                                | 2  |
| 3  | Showcase of the diamond square algorithm. Image taken from Diamond-square algorithm in Wikipedia [30]. . . . .                                                                                                                                                                                                                     | 2  |
| 4  | The Perlin noise algorithm. Image taken from Perlin noise in Wikipedia [29]. . . . .                                                                                                                                                                                                                                               | 3  |
| 5  | A blurred heightmap created with DLA (Left). A real heightmap from a region of Alaska (Right). Images taken from Wikipedia [24] and World of Level Design [16] . . . . .                                                                                                                                                           | 4  |
| 6  | Showcase of the creation of triangular patches (on the left) and an example of the marching cubes algorithm with infinite procedural terrain (on the right). Images taken from Wikipedia [31] and GPU Gems 3 [6]. . . . .                                                                                                          | 4  |
| 7  | Illustration of the hydraulic erosion system implemented in this thesis. . . . .                                                                                                                                                                                                                                                   | 6  |
| 8  | Entity Component System setup in pyGANDALF [20]. . . . .                                                                                                                                                                                                                                                                           | 7  |
| 9  | A terrain rendering created in Terragen [21]. . . . .                                                                                                                                                                                                                                                                              | 8  |
| 10 | A terrain created with Gaia in Unity [33]. . . . .                                                                                                                                                                                                                                                                                 | 9  |
| 11 | A planetary landscape in No man's sky [5]. . . . .                                                                                                                                                                                                                                                                                 | 9  |
| 12 | Comparison between purely random noise (left) and a structured heightmap generated using Perlin noise (right) with parameters $seed = 1$ and $frequency = 1$ . . . . .                                                                                                                                                             | 11 |
| 13 | The same heightmap as in Figure 12, but with a reduced frequency of 0.8. The result appears as a zoomed-in version of the previous map. . . . .                                                                                                                                                                                    | 11 |
| 14 | Enhancing the heightmap with one additional octave of noise introduces greater detail. In this example, the <i>lacunarity</i> parameter is set to 2 and the <i>persistence</i> parameter to 0.5. . . . .                                                                                                                           | 11 |
| 15 | The second heightmap is derived from the first by applying ridge shaping with a strength parameter of 3. The third demonstrates the effect of turbulent noise applied to the original heightmap. In the final example, both techniques are combined, using turbulent noise and ridge shaping with a reduced strength of 2. . . . . | 12 |

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                      |    |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 16 | Heightmap generated with 12 octaves, a <i>frequency</i> of 0.5, <i>lacunarity</i> set to 2, <i>persistence</i> of 0.5, fall-off height of $-0.78$ , and rectangular fall-off using parameters $a = 4.4$ and $b = 0.37$ . In the subsequent heightmap, the value of $b$ is reduced to 0.2, resulting in a smaller area, while in the final example, $a$ is increased to 10.6, producing a sharper transition at the boundary. . . . . | 13 |
| 17 | The coordinates of the grid without the patches. Image taken from LearnOpenGL Tessellation Chapter I: Rendering Terrain using Height Maps [17] . . . . .                                                                                                                                                                                                                                                                             | 14 |
| 18 | In OpenGL there are four different edge values we can set as specified by the built-in GLSL array <code>gl_TessLevelOuter</code> and two internally values with <code>gl_TessLevelInner</code> . Image taken from LearnOpenGL Tessellation Chapter II: Rendering Terrain using Tessellation Shaders & Dynamic Levels of Detail [18] . . . . .                                                                                        | 15 |
| 19 | Showcase of the tessellation (left) and the terrain with the heightmap applied (right). . . . .                                                                                                                                                                                                                                                                                                                                      | 16 |
| 20 | Each channel of the splat map encodes the blending weight for a corresponding terrain texture. Image taken from Polycount Wiki [28].                                                                                                                                                                                                                                                                                                 | 17 |
| 21 | A PBR implementation using the same functions and image based lighting. Image taken from LearnOpenGL PBR: Theory [27]. . . .                                                                                                                                                                                                                                                                                                         | 18 |
| 22 | In the first image, the parameters are set as follows: <i>snowHeight</i> = 0.99, <i>grassHeight</i> = 0.98, and <i>maxHeight</i> = 28.4. In the second image, we increase the <i>maxHeight</i> to 34.7, resulting in more prominent presence of sand. In the third image, we lower the <i>grassHeight</i> to 0.77 to increase grass coverage. . . . .                                                                                | 21 |
| 23 | A texture of sand and a texture of cobblestones being blended using this method. Notice how the sand seems to fall between the stones making the transition much more natural. Image taken from [14].                                                                                                                                                                                                                                | 22 |
| 24 | Comparison between linear interpolation (third from top) and the proposed method (bottom) for blending a rock texture with a grass texture, using a <i>depth</i> parameter of 0.03. The transition in the proposed method appears more realistic, with grass emerging through the cracks of the rock surface. . . . .                                                                                                                | 23 |
| 25 | The repetitive pattern disappears after applying the texture bombing method. . . . .                                                                                                                                                                                                                                                                                                                                                 | 24 |
| 26 | Example of terrain generated and shaded using the proposed system.                                                                                                                                                                                                                                                                                                                                                                   | 24 |

|    |                                                                                                                                                                                                                                                                                |    |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 27 | The terrain from Figure 26 after the application of hydraulic erosion. Simulation parameters: time step $dt = 1.2$ , evaporation rate $= 0.001$ , deposition rate $k_c = 0.1$ , friction coefficient $= 0.05$ , and particle density $k = 1$ . . . . .                         | 26 |
| 28 | Code snippet from the <code>on_update_entity</code> function, responsible for constructing the grid as described in Section 3.2. . . . .                                                                                                                                       | 28 |
| 29 | Terrain generated from a heightmap of Iceland (top), and the result after applying three successive layers of hydraulic erosion (bottom). . . . .                                                                                                                              | 29 |
| 30 | The <code>on_update_entity</code> function in erosion system. . . . .                                                                                                                                                                                                          | 30 |
| 31 | The <code>export_texture</code> function used to extract and save the computed heightmap from the GPU to disk. . . . .                                                                                                                                                         | 31 |
| 32 | The rendered water surface (top), its corresponding reflection texture (middle), and the depth texture used for refraction (bottom). Note that the reflection texture is shown inverted vertically. . . . .                                                                    | 32 |
| 33 | Code snippet from the water fragment shader demonstrating the calculation of normalized device coordinates and the water depth. . . . .                                                                                                                                        | 32 |
| 34 | The red arrow represents the depth of the current water fragment ( <code>gl_FragCoord.z</code> ), the yellow arrow indicates the depth sampled from the refraction depth texture, and the green arrow shows the resulting water depth obtained by subtracting the two. . . . . | 33 |
| 35 | The landscape depicted in Figure 32 rendered without soft edge blending, resulting in visibly harsh transitions between water and terrain. . . . .                                                                                                                             | 33 |
| 36 | A dudv map. We sample this texture to offset the coordinates of the reflection and refraction textures. . . . .                                                                                                                                                                | 34 |
| 37 | Shader code responsible for applying dynamic distortion to the water surface using a DuDv map. . . . .                                                                                                                                                                         | 34 |

## List of Tables

|   |                                                    |    |
|---|----------------------------------------------------|----|
| 1 | Performance of terrain generation system . . . . . | 35 |
| 2 | Performance of erosion system . . . . .            | 35 |



Figure 1: Minecraft, one of the most popular video games, uses procedural generation. Image taken from Minecraft website [22]

## 1 Introduction

The creation of realistic terrains plays a crucial role across various areas such as video game development, film production and education as it enhances immersion within the virtual world. However, this process is often complex and time-consuming, requiring skilled artists with a deep understanding of geological formations to design landscapes accurately. Typically, developers rely on specialized tools, such as Unity’s terrain tools, to construct detailed and visually compelling environments.

### 1.1 Procedural generation

A way to automate this process is with the use of procedural generation which is a fundamental technique in computer graphics, enabling the creation of data algorithmically.

One of the earliest research papers on procedural terrain generation is “Computer Rendering of Stochastic Models” [3] by Fournier, Fussell, and Carpenter (1982), who introduced the use of stochastic subdivision methods to generate natural-looking surfaces (see Figure 2). This method was inspired by fractal geometry, a concept popularized by the mathematician Benoît B. Mandelbrot (1982) in his book *The Fractal Geometry of Nature* [10]. His work provided a mathematical framework for describing self-similar structures found in nature, such as coastlines, mountains, and clouds.

Similarly, Voss, R. (1983), in his paper “Random Fractal Forgeries”, employs fractal-based methods for terrain generation and introduces the concept of  $1/f$  noise, which is inversely proportional to frequency, allowing for a balanced distribution of smooth and rough details [25]. Another widely used and efficient algorithm for generating heightmaps is the Diamond-Square algorithm. This method

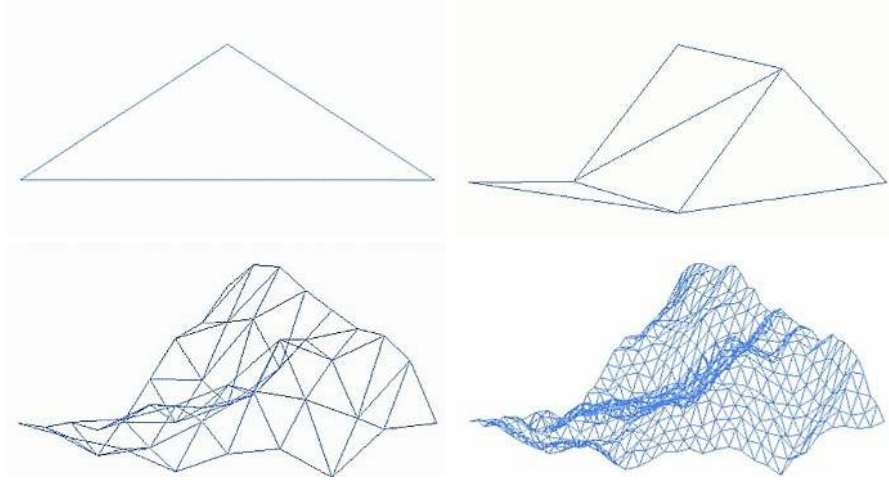


Figure 2: Showcase of the algorithm described in [3].

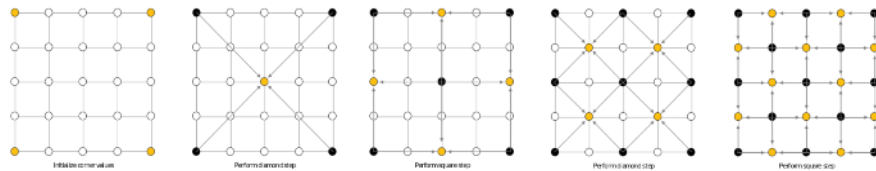


Figure 3: Showcase of the diamond square algorithm. Image taken from Diamond-square algorithm in Wikipedia [30].

was initially introduced in Fournier & Reeves (1983) through their work “A Simple Model of Ocean Wave” [4], and was later extended to terrain generation by Miller, G. S. P. (1986) in “The Definition and Rendering of Terrain Map” [13]. The algorithm initiates with a two-dimensional grid. For each square within the grid, the center point is computed as the average of its four corner points, augmented by a random displacement. Similarly, for each diamond-shaped configuration, the central point is determined by averaging its four surrounding vertices and adding a random displacement, as illustrated in Figure 3. By recursively applying this process, the algorithm generates realistic terrain structures with fractal-like features.

### 1.1.1 The perlin noise technique

In 1983, Ken Perlin sought to develop a more natural-looking noise algorithm after being dissatisfied with the artificial and mechanical appearance of existing techniques. His research, later published in Perlin, K. (1985), “An Image Synthesizer”, introduced Perlin Noise, a gradient-based noise function [19].

This technique establishes a grid wherein each grid point is assigned a pseudo-random gradient vector. For any given coordinate, the algorithm identifies the corresponding grid cell and calculates the dot product between the coordinate vector and the gradient vectors of the cell’s surrounding corners. A smooth fade

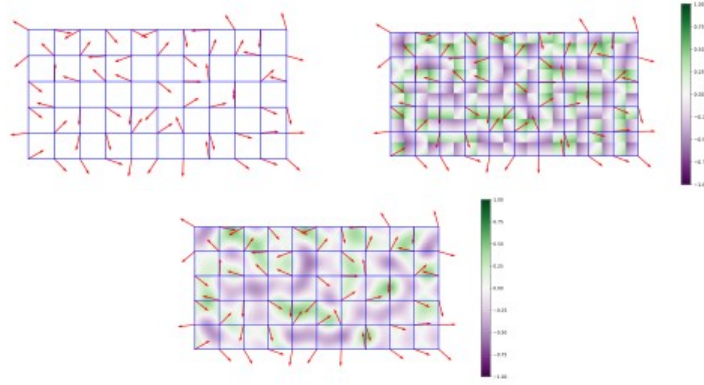


Figure 4: The Perlin noise algorithm. Image taken from Perlin noise in Wikipedia [29].

function is then applied to interpolate these dot product values, resulting in a continuous and coherent noise function (see Figure 4). This method offers an efficient and computationally inexpensive means of generating realistic heightmaps, making it a foundational tool in procedural content generation.

### 1.1.2 Diffusion-Limited Aggregation (DLA)

Another compelling method for generating heightmaps involves the use of Diffusion-Limited Aggregation (DLA), a physical phenomenon wherein particles undergoing random motion adhere upon contact, forming fractal branching structures known as Brownian trees. By associating these particles with pixels on a grid, they can be randomly placed and moved until a collision occurs. While the resulting structure is inherently coarse and block-like, applying a blur filter or using a virtualized grid enables the generation of heightmaps that emulate mountain ridges, as illustrated in Figure 5. This approach to DLA was first introduced by Thomas Witten and Leonard Sander in their seminal 1981 paper, “Diffusion-Limited Aggregation, a Kinetic Critical Phenomenon” [32].

### 1.1.3 Marching cubes

The Marching Cubes algorithm, originally introduced by Lorensen and Cline in their 1987 paper, “Marching Cubes: A High Resolution 3D Surface Construction Algorithm” [9], provides a method for extracting an isosurface mesh from volumetric data. The technique involves dividing the 3D space into a grid of cubes and evaluating each cube to determine whether it contains both a corner with a value below the isovalue and a corner with a value above it. If such a condition is met, the cube is considered to intersect the isosurface. By identifying the edges of the cube that are intersected by the isosurface, triangular patches can be generated to delineate the boundary between the inside and outside of the isosurface. When

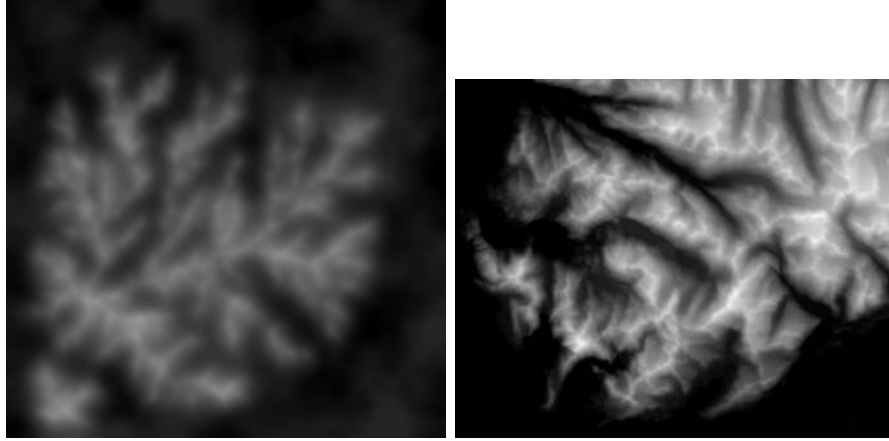


Figure 5: A blurred heightmap created with DLA (Left). A real heightmap from a region of Alaska (Right). Images taken from Wikipedia [24] and World of Level Design [16]

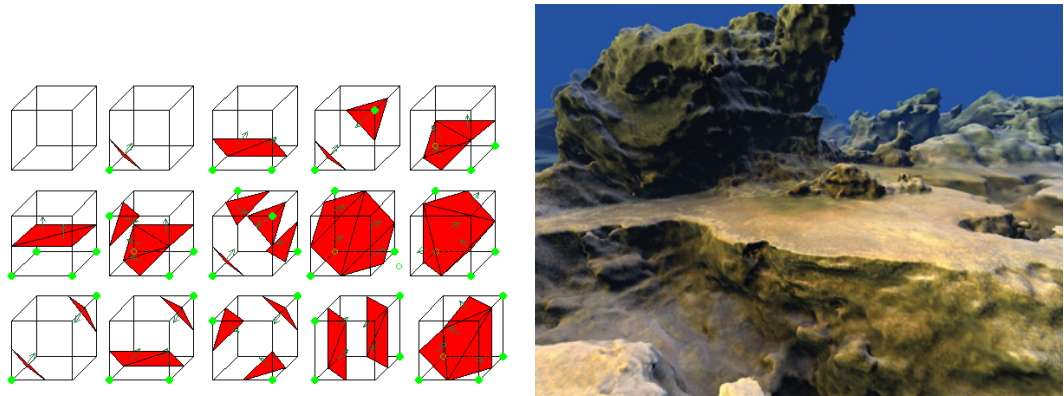


Figure 6: Showcase of the creation of triangular patches (on the left) and an example of the marching cubes algorithm with infinite procedural terrain (on the right). Images taken from Wikipedia [31] and GPU Gems 3 [6].

combined with pseudo-randomly generated 3D textures, this method allows for the creation of volumetric terrains that support complex features such as caves, overhangs, and tunnels—structures that cannot be represented using traditional heightmaps (see Figure 6).

## 1.2 Erosion

The simulation of rain erosion on terrain was initially investigated by Musgrave, Kolb, and Mace (1989) as a means of enhancing the realism of procedurally generated landscapes [15]. Their approach models the descent of raindrops across a heightmap, where each droplet erodes sediment as it traverses lower elevations. Additionally, the model accounts for thermal erosion, where material is gradually broken down due to gravitational slippage, further refining terrain features.

An alternative approach was proposed by Chiba, Muraoka, and Fujita (1998)

in “An Erosion Model Based on Velocity Fields for the Visual Simulation of Mountain Formation” [2]. Their method utilizes velocity fields to dynamically simulate the motion of water and sediment. These velocity fields adapt to the shape of the terrain, leading to realistic results, such as the formation of river valleys and smoothing of the terrain over time. By considering fluid dynamics, this model offers a physically based approach to erosion simulation.

While the aforementioned approaches yield physically accurate results, they are often computationally intensive and not well-suited for real-time applications. To address this limitation, Mei, Decaudin, and Hu (2007) introduced a parallelizable algorithm for real-time hydraulic erosion in their work “Fast Hydraulic Erosion Simulation and Visualization on GPU” [12]. Their method employs a grid-based model executed entirely on the GPU, leveraging its parallel architecture to significantly outperform traditional CPU-based simulations. This advancement enables interactive visualization and real-time terrain erosion, making it suitable for applications such as games and virtual environments.

A different approach to procedural terrain generation is described by G enevaux et al. (2013) in “Terrain Generation Using Procedural Models Based on Hydrology.” that focuses on procedural river network generation, which then serves as the foundation for constructing the elevation map of the terrain [8]. By simulating the natural flow of water, this technique produces highly realistic landscapes that reflect the natural structure of river systems. However, while this method achieves visually convincing results, it cannot accurately simulate river deltas, which require more complex sediment deposition and flow dynamics. Additionally, it is computationally intensive and does not run in real-time, making it less suitable for interactive applications.

Another particle approach is described by Hans Theobald Beyer, “Implementation of a method for hydraulic erosion,” 2015 [1], which models the natural displacement of sediment due to rainfall and water movement over a terrain. The work focuses on realistic terrain modification, incorporating erosion processes such as sediment transport, deposition, and dissolution to create more visually convincing and physically accurate landscapes. It is also worth noting that it encompasses thermal erosion. A significant limitation of this method lies in its computational inefficiency and the inherent difficulty in parallelization. This is primarily due to the fact that each simulated particle must evaluate its surrounding points to determine its direction of movement, resulting in complex data dependencies that hinder parallel execution.

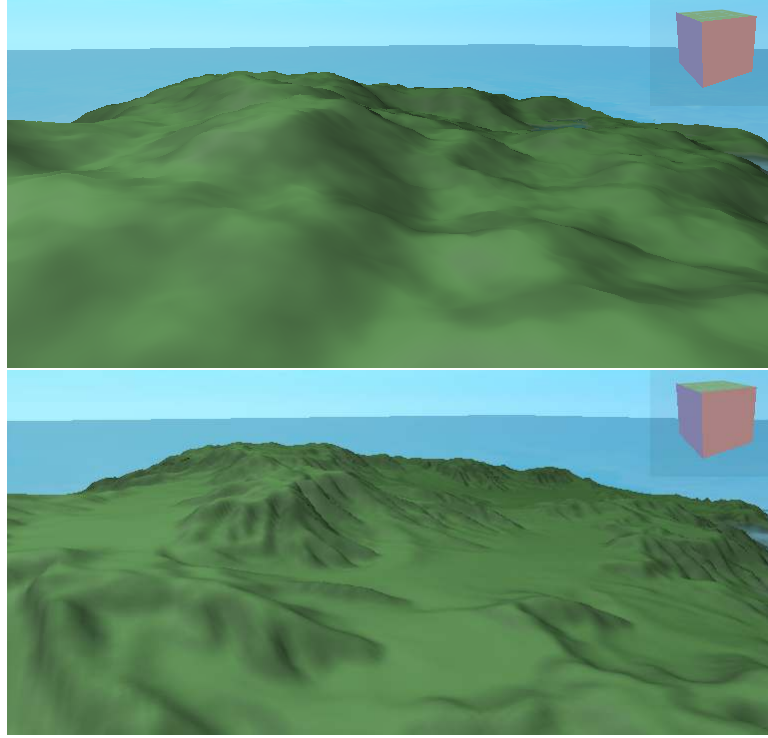


Figure 7: Illustration of the hydraulic erosion system implemented in this thesis.

### 1.3 The pyGANDALF Framework

Teaching computer graphics presents significant challenges due to the inherent complexity of the underlying mathematical foundations and the continuously evolving nature of graphics APIs. Although traditional tools such as OpenGL remain widely utilized, they may not adequately expose students to modern rendering paradigms. To address this limitation, pyGANDALF has been introduced [20] as a Python-based, open-source framework that merges accessibility with support for both legacy and contemporary graphics technologies. A distinguishing feature of the framework is its dual compatibility with OpenGL and WebGPU, enabling students to compare and contrast traditional and modern graphics APIs. This dual support not only enhances conceptual understanding of graphics pipeline evolution but also grants access to advanced features such as compute shaders and ray tracing. Consequently, the framework provides students with practical experience that aligns closely with current industry practices.

The framework leverages the Entity-Component-System (ECS) architecture, a design pattern that promotes modularity and scalability, enabling efficient management of complex scenes and animations. Additionally, pyGANDALF implements advanced rendering techniques, including Physically Based Rendering (PBR), tessellation, geometry shaders, shadow mapping and support for USD formats, making it well-suited for modern computer graphics education. Furthermore, an integrated editor facilitates real-time scene editing and visualization,

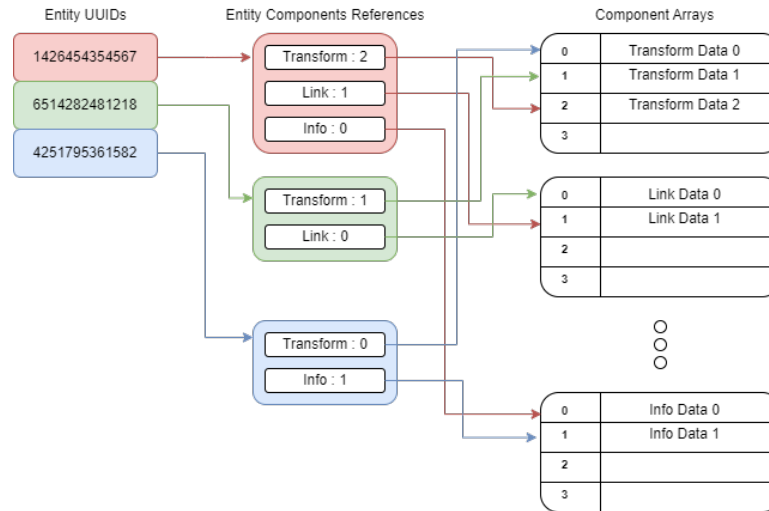


Figure 8: Entity Component System setup in pyGANDALF [20].

enhancing the learning experience.

## 1.4 The Entity-Component-System

The Entity-Component-System (ECS) is a software architectural pattern that decouples data from behavior, adhering to the principle of composition over inheritance. Within this paradigm, entities serve as general-purpose identifiers that are composed of components, which encapsulate all relevant data. Systems operate on entities by processing those that possess the requisite components. This architecture is widely adopted in video game development, where it facilitates the modular and flexible representation of game objects.

Within the pyGANDALF framework, a custom implementation of the Entity-Component-System (ECS) architecture is employed, prioritizing simplicity and clarity to better serve educational objectives. Specifically, the scene is represented as an array of entities, each identified by a Universally Unique Identifier (UUID). Corresponding to each component type, separate arrays are maintained to store component data. A dictionary-based structure is utilized to map components to their respective entities, as illustrated in Figure 8. Upon instantiation, systems specify the component types on which they will operate.

## 1.5 Thesis Overview

The remainder of this thesis is organized into the following six chapters:

- Section 2 reviews the state of the art in procedural terrain generation across various industries and discusses relevant implementations.
- Section 3 presents the algorithms employed for heightmap generation.



Figure 9: A terrain rendering created in Terragen [21].

- Section 4 explores the techniques utilized for terrain shading.
- Section 5 outlines the architecture and methodology behind the erosion simulation.
- Section 6 provides a detailed account of the integration of these algorithms within the pyGANDALF framework.
- Finally, Section 7 concludes the thesis by summarizing the key findings and proposing directions for future research.

The primary objective of this thesis is the development of a tool capable of procedurally generating an infinite variety of terrain, with automated real-time shading and erosion effects.

## 2 State of the Art

Today, there are many applications and tools that focus on realistic procedural terrain generation. An example is Terragen (see Figure 9), which can render a photorealistic environment but does not operate in real time, as it is primarily designed for high quality offline rendering used in film, visual effects, and architectural visualization. A similar tool is Gaia (see Figure 10), a terrain generation tool for Unity that allows users to sculpt, texture, and populate terrains procedurally. While Gaia itself does not generate terrains in real-time at runtime, it can be used to pre-generate procedural terrains within Unity before execution. Some runtime terrain generation features exist, but they are not designed for large-scale real-time terrain generation.

In the video game industry, notable examples of procedural generation include Minecraft, which generates an effectively infinite world as the player explores, using a voxel-based representation (see Figure 1). Another example is No Man's Sky (see Figure 11), which employs procedural algorithms to generate entire



Figure 10: A terrain created with Gaia in Unity [33].



Figure 11: A planetary landscape in No man's sky [5].

planets. Although these planets are not infinite, they are immense in scale and allow for seamless exploration.

Additionally, recent research has explored the use of Generative Adversarial Networks (GANs) for procedural terrain generation. In the work titled “Procedural Terrain Generation Using Generative Adversarial Network” by Georgios Voulgaris, Ioannis Mademlis, and Ioannis Pitas [26], the authors demonstrate that by training neural models on real-world heightmap data, it is possible to generate realistic terrain images.

### 3 Procedural Terrain Generation

In this thesis, terrain generation is approached using a heightmap-based method. Initially, a uniform grid of points is constructed, and for each point in this grid, a scalar value representing elevation must be assigned. A naive strategy would involve generating a pseudo-random value for each point; however, this results in a surface that resembles static noise and lacks spatial coherence. To produce a heightmap that more accurately resembles natural terrain, a noise function with smoother transitions and local correlation is required. For this purpose, we employ Perlin noise [19], a technique belonging to the category of gradient noise.

A detailed explanation of this method is provided in Section 1.1.1.

### 3.1 Heightmap

By evaluating the Perlin noise function at each 2D coordinate of the grid and storing the resulting value across all RGB channels of a 2D texture, a grayscale representation of the heightmap is obtained (see Figure 12). It is important to emphasize that the Perlin noise function is deterministic; that is, it returns the same value for a given input coordinate. To introduce variability and generate different terrain configurations, a seed-based offset is applied to the input coordinates prior to sampling the noise function. This offset ensures that different outputs can be produced while preserving the smooth characteristics of the noise:

$$height = \text{PerlinNoise}(x + seed \cdot 0.83, z + seed \cdot 23.097)$$

Further variation can be achieved by scaling the *seed* with additional constants. Assuming the grid lies in the  $x$ - $z$  plane, the final 3D position of a point on the terrain can be expressed as:

$$point = (x, height, z)$$

An essential feature of the heightmap generation process is the ability to control the scale of the noise. This is achieved through the use of a *frequency* parameter, which determines the spacing between the sampled points by scaling the  $x$  and  $z$  coordinates (see Equation 1). Higher frequency values result in more rapid changes in elevation, producing finer details, whereas lower values yield smoother, more gradual terrain variations (see Figure 13). For more intuitive control and to allow for continuous scaling, the frequency can be divided by a constant factor.

$$height = \text{PerlinNoise}\left(\frac{x \cdot frequency}{64} + seed \cdot 0.83, \frac{z \cdot frequency}{64} + seed \cdot 23.097\right) \quad (1)$$

As illustrated by the generated textures, the resemblance to realistic mountainous terrain and ridgelines remains insufficient. A widely adopted technique to improve realism is the use of fractal noise, which involves combining multiple layers of noise—referred to as *octaves*—at varying frequencies as shown in Figure 14. Each successive octave contributes less to the overall heightmap. This diminishing influence is controlled by the *amplitude* parameter, which scales the current height value before adding it to the cumulative result. Initially set to 1,

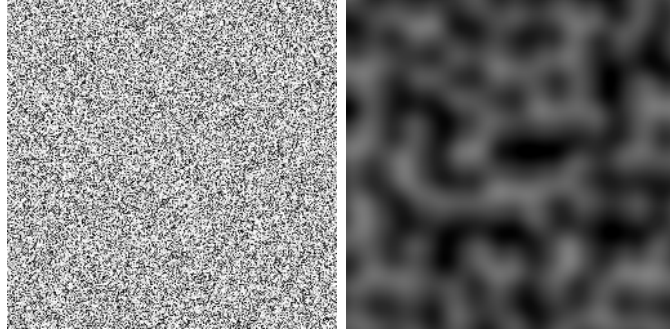


Figure 12: Comparison between purely random noise (left) and a structured heightmap generated using Perlin noise (right) with parameters  $seed = 1$  and  $frequency = 1$ .

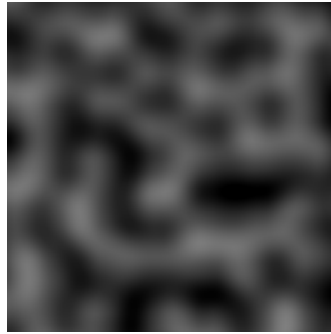


Figure 13: The same heightmap as in Figure 12, but with a reduced frequency of 0.8. The result appears as a zoomed-in version of the previous map.

the amplitude is reduced in each iteration by a factor defined by the *persistence* parameter. Similarly, the *frequency* increases with each octave, scaled by the *lacunarity* parameter. Decreasing both parameters results in a smoother terrain surface. Conversely, increasing the *lacunarity* introduces finer details resembling rocky structures, while a higher *persistence* amplifies the vertical intensity of these features.

A notable variation of noise, referred to by Perlin as turbulent noise, involves summing the absolute values of each octave. Since standard noise functions return values within the range  $[-1, 1]$ , taking the absolute value effectively remaps this

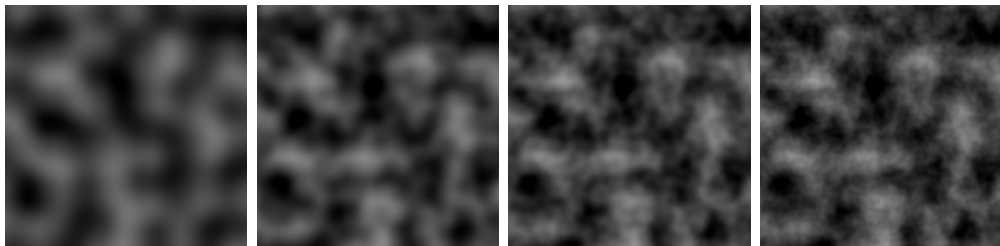


Figure 14: Enhancing the heightmap with one additional octave of noise introduces greater detail. In this example, the *lacunarity* parameter is set to 2 and the *persistence* parameter to 0.5.

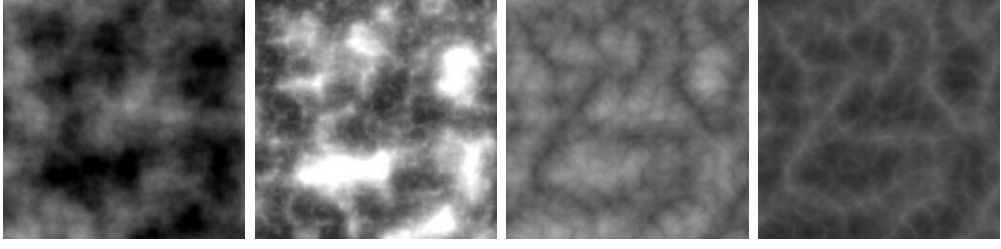


Figure 15: The second heightmap is derived from the first by applying ridge shaping with a strength parameter of 3. The third demonstrates the effect of turbulent noise applied to the original heightmap. In the final example, both techniques are combined, using turbulent noise and ridge shaping with a reduced strength of 2.

to  $[0, 1]$ . The resulting patterns resemble natural phenomena such as clouds, and are particularly useful for generating hills and canyons (see third image of Figure 15).

To further enhance the appearance of mountainous ridges, one can apply the transformation:

$$(1 - height)^{strength}$$

This function sharpens elevation transitions, as the derivative of the exponential term increases with the exponent. The parameter *strength* allows for control over the steepness of the terrain, enabling the simulation of sharp ridges and abrupt elevation changes like in Figure 15.

By defining a geometric shape (such as a circle or rectangle) centered on the map, it is possible to gradually decrease the terrain height outward until it reaches zero beyond the boundary of this shape, effectively creating an island. This is achieved by performing a linear interpolation between the existing height at each coordinate and a predefined constant height (fall-off height). The interpolation factor is computed based on the distance of the point from the center of the map, which is derived using the equation of a circle (see Equation 2) or a square (see Equation 3). To ensure the interpolation factor remains within the range  $[0, 1]$ , all coordinates must be normalized accordingly. Given that the coordinates span from 0 to  $mapsize - 1$ , normalization is performed as follows:

$$x = \frac{x}{mapsize - 1}, \quad z = \frac{z}{mapsize - 1}$$

Within this normalized coordinate space, the center of the map corresponds to the point  $(0.5, 0.5)$ . Therefore, the equation used to compute the squared distance from the center becomes:

$$distance = (x - 0.5)^2 + (z - 0.5)^2 \quad (2)$$



Figure 16: Heightmap generated with 12 octaves, a *frequency* of 0.5, *lacunarity* set to 2, *persistence* of 0.5, fall-off height of  $-0.78$ , and rectangular fall-off using parameters  $a = 4.4$  and  $b = 0.37$ . In the subsequent heightmap, the value of  $b$  is reduced to 0.2, resulting in a smaller area, while in the final example,  $a$  is increased to 10.6, producing a sharper transition at the boundary.

for a circle, and for a square:

$$distance = \max(|(x \cdot 2) - 1|, |(z - 2) - 1|) \quad (3)$$

To introduce user-defined control over the fall-off behavior, the interpolation value is computed using the following function:

$$interpValue = \frac{distance^a}{distance^b + (b - b \cdot distance)^a}$$

where  $a$  and  $b$  are adjustable parameters that influence the shape of the interpolation curve. As illustrated in Figure 16, the parameter  $b$  controls the scale at which the fall-off occurs, while  $a$  affects the sharpness of the transition. Smaller values of  $a$  result in a smoother gradient between the original heightmap and the fall-off region.

### 3.2 Tessellation

Tessellation refers to the process of subdividing geometric primitives directly on the GPU to dynamically enhance detail. This functionality is supported by several modern graphics APIs as part of their rendering pipelines. In OpenGL, tessellation is executed between the vertex and geometry shader stages and consists of two main phases. The first phase involves the tessellation control shader, which specifies the tessellation levels and parameters. Subsequently, the tessellation evaluation shader computes the positions of the newly generated vertices. This method is particularly well-suited for terrain rendering, as it allows for the addition of geometric detail without the need to reconstruct or modify the original grid structure.

When implementing tessellation in OpenGL, it is necessary to use the *patches*

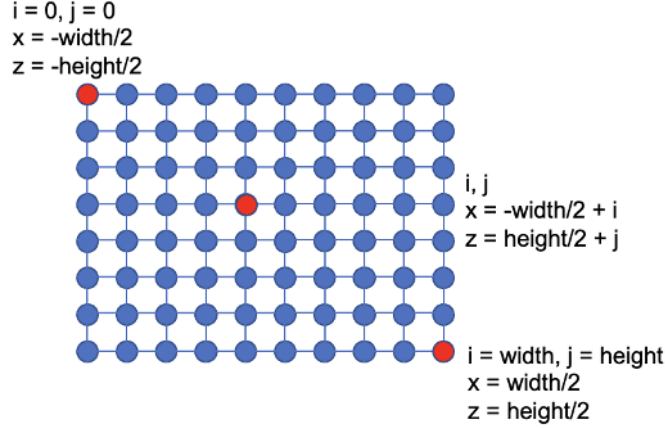


Figure 17: The coordinates of the grid without the patches. Image taken from LearnOpenGL Tessellation Chapter I: Rendering Terrain using Height Maps [17]

primitive and to define the number of vertices that each patch will contain. In the context of terrain rendering, the surface is divided into a grid of square patches, each composed of four vertices. The *resolution* variable denotes the number of patches along both the horizontal and vertical directions, resulting in a total of  $\text{resolution} \cdot \text{resolution}$  patches. The mesh is centered at the origin of the coordinate system (see Figure 17). Consequently, the  $x$ - and  $z$ -coordinates of the top-left vertex of a patch at grid position  $(i, j)$  are computed as follows:

$$\left( -\frac{\text{mapsize}}{2} + \frac{\text{mapsize} \cdot i}{\text{resolution}}, \quad -\frac{\text{mapsize}}{2} + \frac{\text{mapsize} \cdot j}{\text{resolution}} \right)$$

Here,  $i$  denotes the row index and  $j$  the column index of the current patch. To compute the coordinates of the remaining vertices of the patch, the indices are adjusted as follows: for the top-right vertex, increment  $j$  by 1; for the bottom-left vertex, increment  $i$  by 1; and for the bottom-right vertex, increment both  $i$  and  $j$  by 1. The corresponding texture coordinates are normalized in the range  $[0, 1]$  by dividing the patch indices by the total resolution.

The tessellation control shader is executed once for each vertex in a patch and is responsible for specifying the tessellation levels. Tessellation is performed by subdividing the edges of the patch, generating additional vertices along the lines connecting the original vertices. Therefore, it is necessary to define how many segments each edge of the patch should be divided into.

To achieve a dynamic level of detail, the tessellation factor must depend on the distance between the camera and each vertex. This can be accomplished by transforming the vertex into camera space using the model and view matrices and extracting the absolute value of its  $z$ -coordinate. A minimum and maximum distance threshold are defined to control the tessellation levels. The maximum tessellation is applied when the vertex is close to the camera, and the minimum

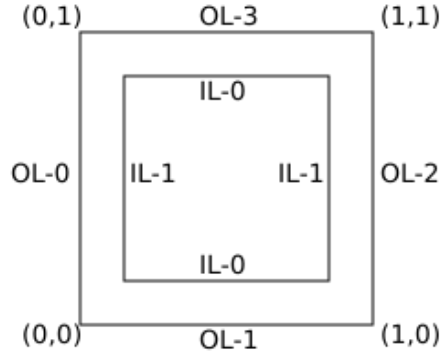


Figure 18: In OpenGL there are four different edge values we can set as specified by the built-in GLSL array `gl_TessLevelOuter` and two internally values with `gl_TessLevelInner`. Image taken from LearnOpenGL Tessellation Chapter II: Rendering Terrain using Tessellation Shaders & Dynamic Levels of Detail [18]

tessellation is applied at the farthest allowable distance. By normalizing the distance value within the defined range, it is possible to interpolate between the maximum and minimum tessellation levels. The tessellation levels, as illustrated in Figure 18, are defined as follows:

$$OL_0 = \text{lerp}(MAX\_TESS, MIN\_TESS, \min(distance_{00}, distance_{01}))$$

$$OL_1 = \text{lerp}(MAX\_TESS, MIN\_TESS, \min(distance_{00}, distance_{10}))$$

$$OL_2 = \text{lerp}(MAX\_TESS, MIN\_TESS, \min(distance_{10}, distance_{11}))$$

$$OL_3 = \text{lerp}(MAX\_TESS, MIN\_TESS, \min(distance_{01}, distance_{11}))$$

$$IL_0 = \max(OL_0, OL_2)$$

$$IL_1 = \max(OL_1, OL_3)$$

For each outer edge, the minimum distance between the two associated vertices is used to determine the tessellation level via interpolation. The inner tessellation levels are then computed as the maximum of their corresponding adjacent outer levels.

The purpose of the tessellation evaluation shader is to compute the position and texture coordinates of the vertices generated during the tessellation process. This is accomplished through bilinear interpolation of the positions and texture coordinates of the four vertices defining the patch, using the built-in variable `gl_TessCoord`, which provides the barycentric coordinates for the generated point within the patch. The interpolated texture coordinates are then used to sample the heightmap to obtain the elevation of the point. Additionally, adjacent pixels

(top, bottom, left, and right) are sampled in order to estimate the surface normal, which is essential for lighting calculations. Once the height and normal are computed, the final point is transformed using the model, view, and projection matrices to obtain its position in clip space. The resulting level of detail through tessellation is illustrated in Figure 19.

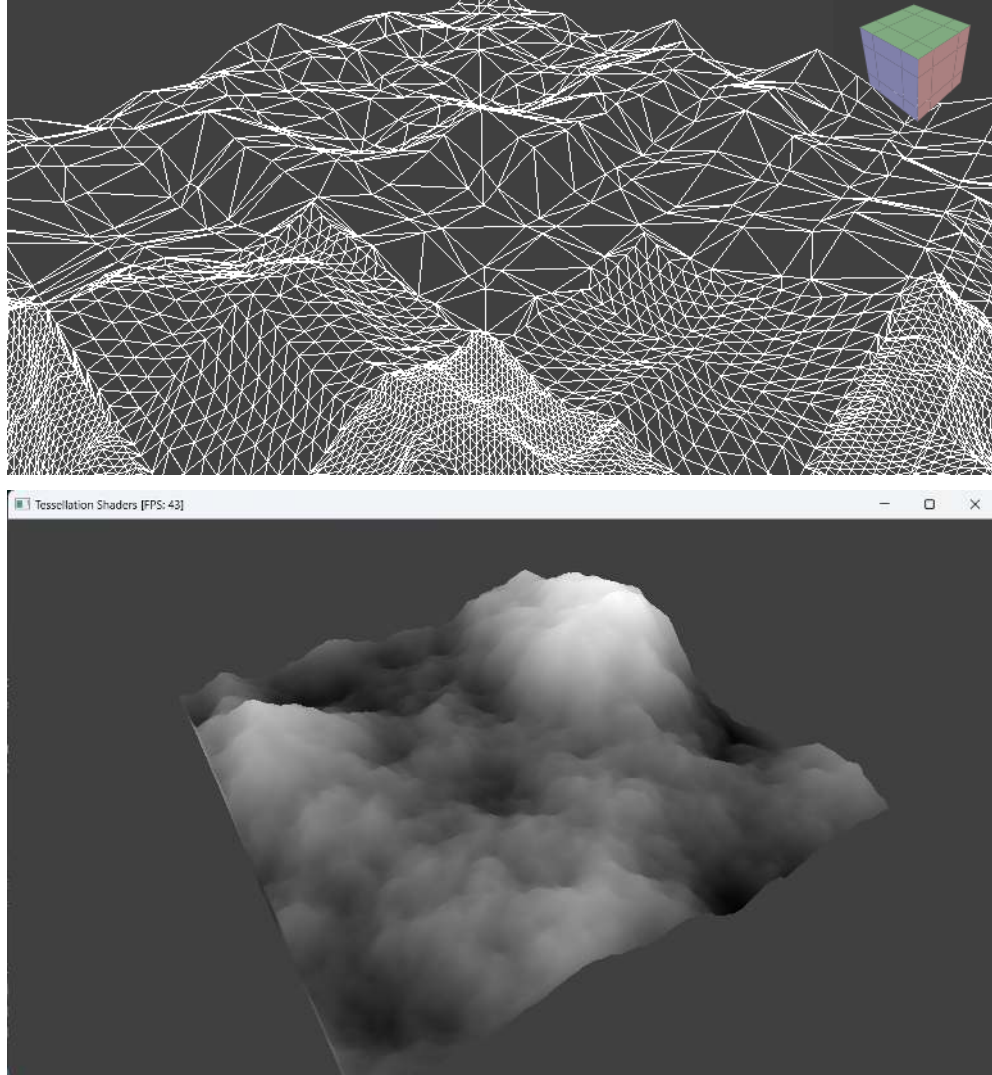


Figure 19: Showcase of the tessellation (left) and the terrain with the heightmap applied (right).

## 4 Shading

The shading of the terrain is a critical factor in enhancing visual realism. Traditionally, this process is performed by artists who design textures corresponding to various terrain features such as rock, sand, and grass. These textures are typically blended using a splat map, a texture in which each channel represents an alpha value that dictates the contribution of a specific terrain texture, as illustrated in

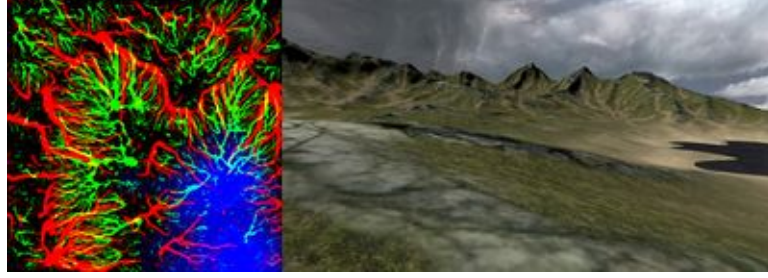


Figure 20: Each channel of the splat map encodes the blending weight for a corresponding terrain texture. Image taken from Polycount Wiki [28].

Figure 20.

In the context of this thesis, where terrain generation is performed in real-time, manual texture painting using splat maps is not feasible. Instead, an automated shading technique is employed in which the contribution of each texture at a given vertex is dynamically computed based on the terrain’s height and slope. This approach ensures that the shading adapts procedurally to the geometry of the terrain, supporting a fully real-time pipeline.

## 4.1 Physically Based Rendering

Firstly, simulating lighting is essential for achieving visual realism. One widely adopted technique is Physically Based Rendering (PBR), a collection of methods that approximates the reflectance equation:

$$L_o(p, \omega_o) = \int_{\Omega} f_r(p, \omega_i, \omega_o) L_i(p, \omega_i) n \omega_i d\omega_i$$

This equation models the outgoing radiance  $L_o$  at point  $p$  in direction  $\omega_o$  as the integral over the hemisphere  $\Omega$  of the incoming radiance  $L_i$ , modulated by the bidirectional reflectance distribution function (BRDF)  $f_r$ , the surface normal  $n$ , and the incoming direction  $\omega_i$ .

PBR leverages the microfacet model, the principle of energy conservation, and physically-based BRDFs to simulate surface reflectance. The microfacet theory posits that surfaces consist of many tiny facets with varying orientations. The roughness of a surface determines the distribution of these facets: rougher surfaces scatter light more diffusely, whereas smoother surfaces reflect light more directly. As such, surface roughness directly influences the material’s reflectivity.

To ensure that PBR adheres to the principle of energy conservation, the incoming light must be partitioned into two components: reflection and refraction. The reflection component corresponds to specular lighting, representing light that reflects directly off the surface. The refraction component corresponds to diffuse



Figure 21: A PBR implementation using the same functions and image based lighting. Image taken from LearnOpenGL PBR: Theory [27].

lighting, representing light that penetrates the surface, scatters internally, and then exits in various directions. This separation ensures that the total reflected and refracted energy does not exceed the incoming energy, preserving physical plausibility.

The BRDF is a core component of Physically Based Rendering (PBR). It models how light is reflected at an opaque surface by taking into account the incoming light direction  $\omega_i$ , the outgoing (view) direction  $\omega_o$ , the surface normal  $\mathbf{n}$ , and a material-dependent surface roughness parameter  $a$ , which characterizes the micro-surface structure. The BRDF determines the proportion of light that is reflected in a specific direction, given these variables. Among various BRDF models, this work employs the Cook-Torrance BRDF, defined as:

$$f_r = k_d \frac{c}{\pi} + k_s \frac{DFG}{4(\omega_o n)(\omega_i n)}$$

Here,  $k_d$  and  $k_s$  represent the diffuse and specular reflection coefficients, respectively, while  $D$ ,  $F$ , and  $G$  denote the Normal Distribution Function, the Fresnel term, and the Geometry function. In this implementation, we adopt the Trowbridge-Reitz GGX distribution for  $D$ , the Fresnel-Schlick approximation for  $F$ , and Smith's Schlick-GGX method for  $G$ . An example of a similar implementation is illustrated in Figure 21.

## 4.2 Height and Slope Blending

In this thesis, a system is implemented to automatically shade the terrain based on the heightmap using three distinct textures: one representing high altitudes (snow), one for intermediate elevations (grass), and one for lower regions (sand). To blend these textures smoothly across the terrain, a hierarchical linear interpolation approach is applied (see Equations 4, 5 and 6). First, a linear interpolation is performed between the grass and snow textures based on elevation. Similarly, another interpolation is performed between the sand and grass textures. Finally, the two interpolated results are blended together using a third linear interpolation step.

$$\text{sandGrass} = \text{lerp}(\text{sand}, \text{grass}, \text{sandToGrassFactor}) \quad (4)$$

$$\text{grassSnow} = \text{lerp}(\text{grass}, \text{snow}, \text{grassToSnowFactor}) \quad (5)$$

$$\text{finalColor} = \text{lerp}(\text{sandGrass}, \text{grassSnow}, \text{finalFactor}) \quad (6)$$

To compute the interpolation factors for texture blending, the terrain height is first normalized to the range  $[0, 1]$  using inverse linear interpolation, with the bounds set to 0 and a user-defined parameter *maxHeight* (see Equation 7). This normalized height value is then further transformed across three distinct ranges to determine the contribution of each texture (see Equations 8, 9 and 10). Two key thresholds are defined: *grassHeight*, indicating the elevation where the grass texture begins to appear, and *snowHeight*, marking the start of the snow texture. The blending regions are thus segmented as follows: heights from 0 to *grassHeight* correspond to the sand texture; heights between *grassHeight* and *snowHeight* correspond to the grass texture; and heights from *snowHeight* to 1 correspond to the snow texture.

$$\text{heightInRange} = \text{invLerp}(0, \text{maxHeight}, \text{height}) = \frac{\text{height}}{\text{maxHeight}} \quad (7)$$

$$\text{grassToSnowFactor} = \text{invLerp}(\text{snowHeight}, 1, \text{heightInRange}) \quad (8)$$

$$\text{sandToGrassFactor} = \text{invLerp}(0, \text{grassHeight}, \text{heightInRange}) \quad (9)$$

$$\text{finalFactor} = \text{invLerp}(\text{grassHeight}, \text{snowHeight}, \text{heightInRange}) \quad (10)$$

The parameter *snowHeight* determines the elevation at which the transition between grass and snow textures occurs, while *grassHeight* controls the blending boundary between sand and grass. The *maxHeight* parameter influences both transitions by scaling the overall height range. Figure 22 illustrates the visual impact of adjusting these parameters. In the second image, both the sand-grass and grass-snow boundaries shift upwards due to an increase in *maxHeight*. In

the third image, only the sand-grass transition lowers, a result of decreasing *grassHeight*, while *snowHeight* remains unchanged.

Finally, the *finalColor* is blended with a rock texture based on the slope of the terrain. The blending factor is computed using the *y*-component of the surface normal, which reflects the steepness of the terrain. Specifically, we subtract this value from 1 and normalize it using the *slopeThreshold* parameter, which defines the slope sensitivity of the blend. This parameter should lie within the range  $[0, 1]$  and allows control over the point at which the rock texture starts to appear. The blending factor is calculated as:

$$rockBlendFactor = 1 - \frac{1 - normalY}{slopeThreshold}$$

#### 4.2.1 Blending

Instead of applying linear interpolation for texture blending, a more advanced method can be employed using the alpha channel of the textures.

For each pair of pixels to be blended, we compute the average of their RGB values and store this result in the alpha channel. We also incorporate an opacity value into the alpha channel, derived from the interpolation factor previously calculated for one of the pixels, and its complement (i.e.,  $1 - \text{factor}$ ) for the other. The final blending operation takes the maximum of the two alpha values and subtracts a user-defined parameter, *depth*, which controls the smoothness of the transition between textures:

$$ma = \max(\text{Color1}_\alpha + 1 - \text{lerpFactor}, \text{Color2}_\alpha + \text{lerpFactor}) - \text{depth}$$

To determine the contribution of each pixel to the final color, we subtract this value from the respective alpha-based weights. If the result is negative, it is clamped to zero:

$$\begin{aligned} b_1 &= \max(\text{Color1}_\alpha + 1 - \text{lerpFactor} - ma, 0) \\ b_2 &= \max(\text{Color2}_\alpha + \text{lerpFactor} - ma, 0) \end{aligned}$$

The final blended color is then computed using a weighted average of the two colors (see Equation 11). The results of this technique are illustrated in Figure 24.

$$\text{FinalColor} = \frac{\text{Color1} \cdot b_1 + \text{Color2} \cdot b_2}{b_1 + b_2} \quad (11)$$

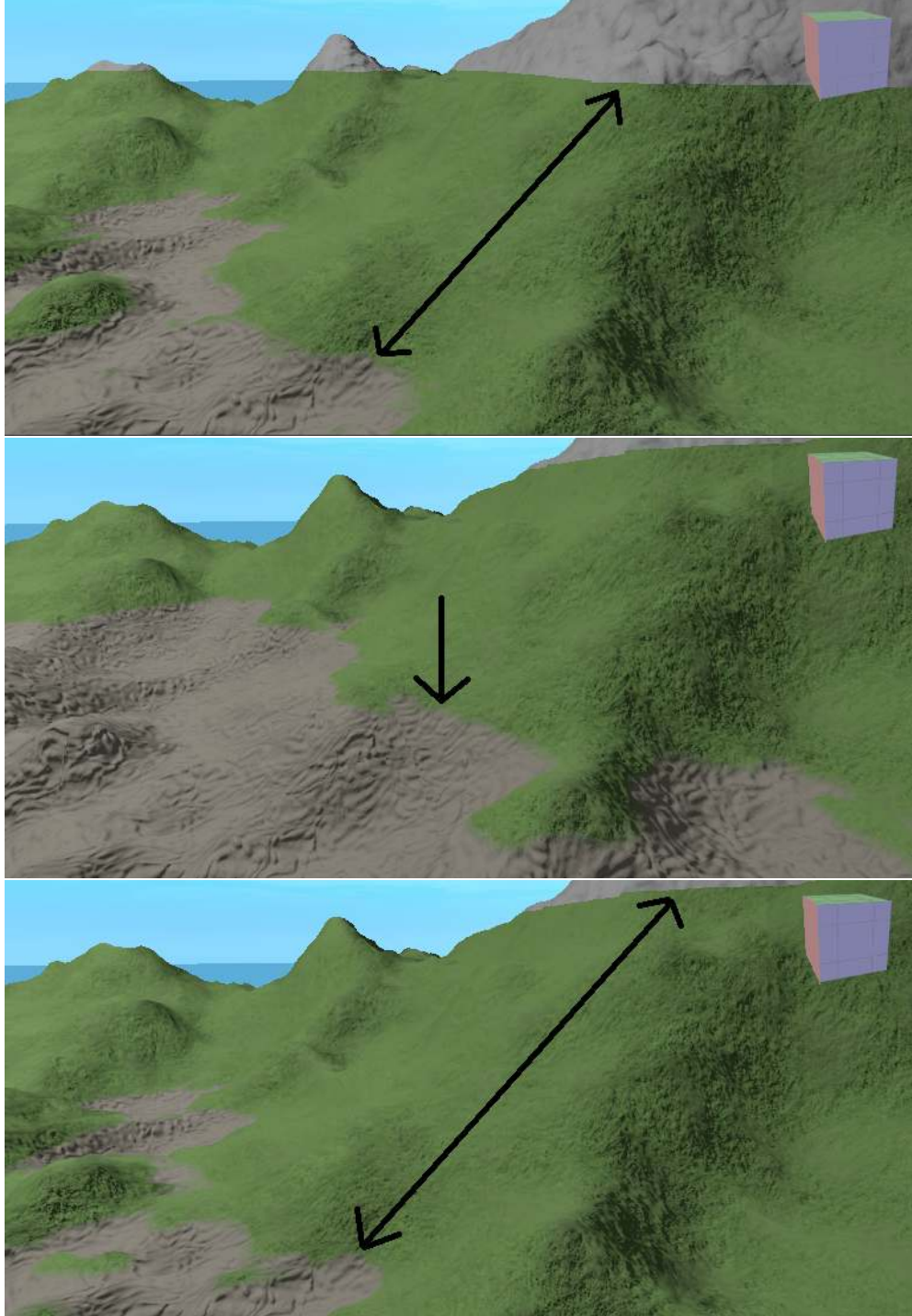


Figure 22: In the first image, the parameters are set as follows:  $snowHeight = 0.99$ ,  $grassHeight = 0.98$ , and  $maxHeight = 28.4$ . In the second image, we increase the  $maxHeight$  to 34.7, resulting in more prominent presence of sand. In the third image, we lower the  $grassHeight$  to 0.77 to increase grass coverage.



Figure 23: A texture of sand and a texture of cobblestones being blended using this method. Notice how the sand seems to fall between the stones making the transition much more natural. Image taken from [14].

#### 4.2.2 Texture Bombing

A common problem that we face in shading is the repetitive pattern that emerges when we tile the textures. Because the terrain mesh is very large and the textures are small, in order to increase the detail, we tile them. This means that the texture is scaled down and then repeated multiple times in the terrain.

Texture bombing is a technique described in NVIDIA’s *GPU Gems* series, specifically in Chapter 20 titled “Texture Bombing” by R. Steven Glanville [7], which addresses the issue of repetitive texture patterns in tiled surfaces. The method divides the UV space into a regular grid, and places randomized texture samples within each cell to reduce visible tiling artifacts. In this implementation, three additional grids are created, each offset by half the grid cell width (0.5). For each of these grids, a random value is computed and added to the original texture UVs. The texture is then sampled using these four modified UV coordinates, and the results are blended based on proximity to the center of each grid cell. The result of this technique is illustrated in Figure 25.

One of the main drawbacks of the texture bombing technique is its performance cost, as it requires four separate texture samples for each fragment, significantly increasing the computational load. Additionally, since the sampled textures are blended together to eliminate visible seams and tiling, the final result can appear slightly blurred, potentially reducing the perceived sharpness of surface details.

## 5 Erosion Simulation

As illustrated in Figure 26, the terrain generated so far exhibits notable limitations in realism. Depending on the frequency of the noise used, the terrain either appears overly mountainous—with an abundance of high-frequency features—or

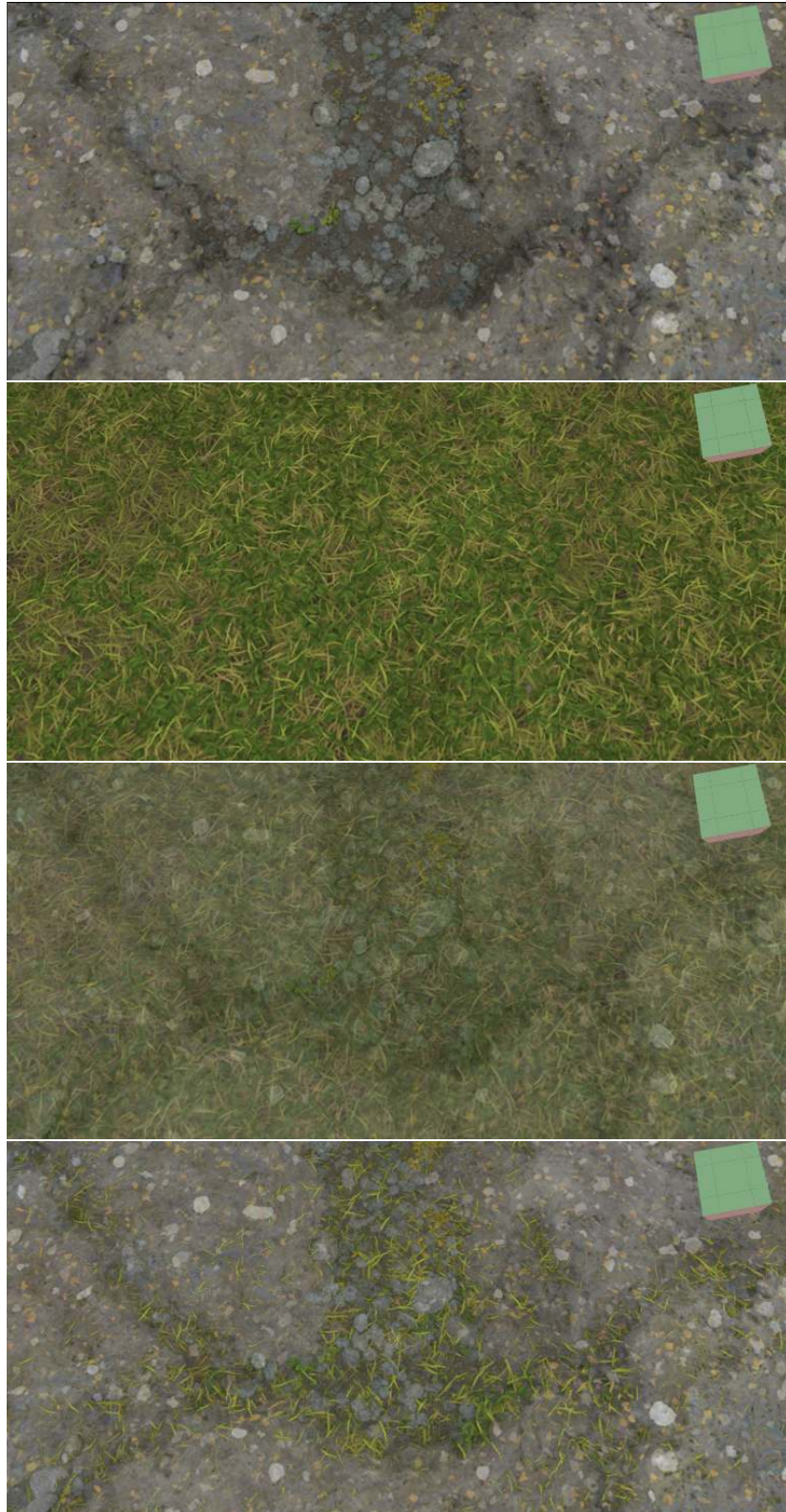


Figure 24: Comparison between linear interpolation (third from top) and the proposed method (bottom) for blending a rock texture with a grass texture, using a *depth* parameter of 0.03. The transition in the proposed method appears more realistic, with grass emerging through the cracks of the rock surface.

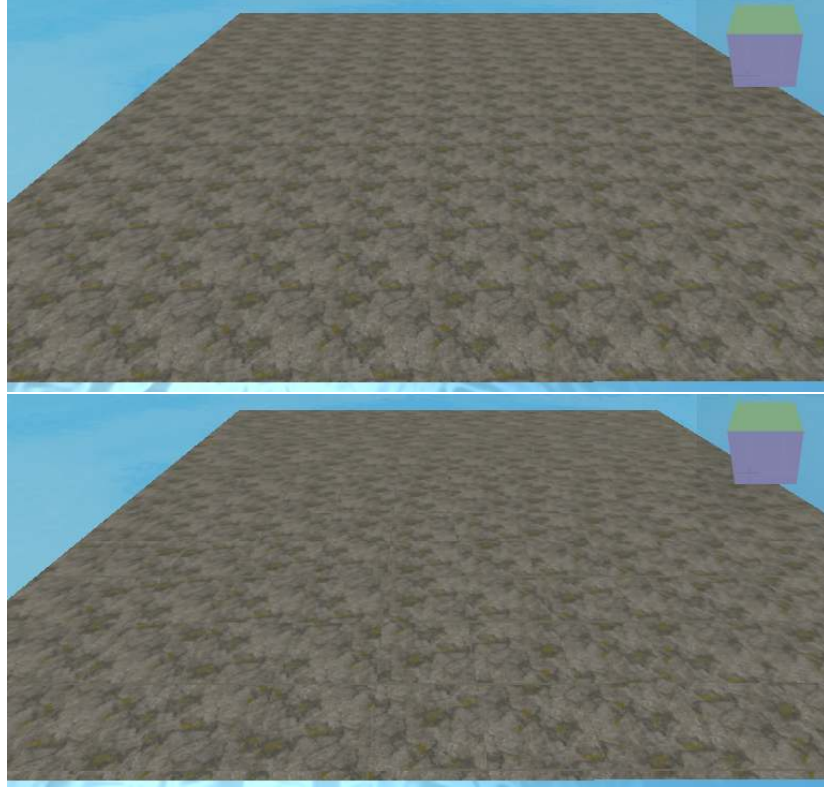


Figure 25: The repetitive pattern disappears after applying the texture bombing method.

overly smooth and hill-like when the frequency is low. This results in a lack of geomorphological diversity. The presence of features such as valleys, riverbeds, cliffs, flatlands, and plateaus is essential for producing terrain that appears natural and believable.

Hydraulic erosion simulation models the effects of rainfall on a heightmap and is a critical technique for producing realistic terrain, as real-world landscapes have been shaped by hydraulic erosion over millions of years. As discussed in Section 1.2, numerous algorithms exist that approximate this phenomenon; how-

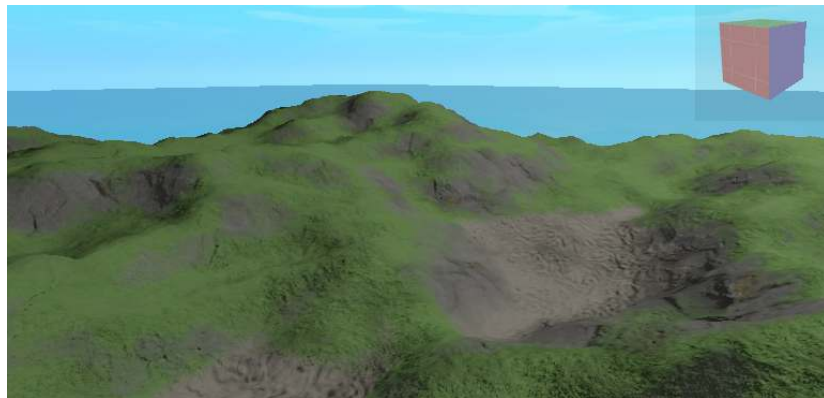


Figure 26: Example of terrain generated and shaded using the proposed system.

ever, many of them suffer from poor performance. Initially, the objective of this thesis was to implement the method proposed by Hans Theobald Beyer [1] using compute shaders. Nevertheless, this approach proved to be highly challenging due to the algorithm’s inherent difficulty in being parallelized. Consequently, the focus shifted toward a simpler, particle-based method described by Nicholas McDonald [11], which, while easier to parallelize, does not incorporate thermal erosion effects.

In this method, each particle is characterized by its position, velocity, volume (representing mass), and sediment load, which corresponds to the amount of sediment it carries as a fraction of its volume. Particles are initially spawned uniformly across the terrain grid, with one particle per grid point. Their positions are updated based on classical mechanics. The particle’s local surface normal is first estimated by sampling neighboring height values to construct directional vectors—typically from left to right and top to bottom—and computing the cross product of these vectors. The resulting surface normal’s  $x$  and  $z$  components determine the direction of the gravitational force acting on the particle. Acceleration is then calculated using Newton’s Second Law, scaled by a tunable parameter  $k$ , as shown in Equation 12. The new velocity and position of the particle are subsequently computed using Equations 13 and 14, respectively.

$$a = \frac{k}{m} \cdot n \quad (12)$$

$$v_{new} = v_{old} + dt \cdot a \quad (13)$$

$$p_{new} = p_{old} + dt \cdot v_{new} \quad (14)$$

To account for surface resistance, velocity is reduced based on terrain friction, as described in Equation 15, where the *friction* coefficient is user-defined. If a particle moves outside the terrain boundaries, the simulation for that particle is terminated.

$$v_{new} = v_{new} \cdot (1 - dt \cdot friction) \quad (15)$$

As the particles traverse the terrain, they interact with it by either removing or depositing sediment. This sediment exchange can be modeled using mass transfer coefficients. Specifically, the transfer of mass between the terrain and the droplet is proportional to the difference between the droplet’s current sediment concentration  $c$  and its equilibrium concentration  $c_{eq}$ , as defined in Equation 16. The parameter  $k_c$  controls the deposition or erosion rate. To ensure physical plausibility, it is essential to define an appropriate model for the equilibrium concentration  $c_{eq}$ . This quantity increases when the particle moves downhill—favoring erosion—and drops to zero when moving uphill—favoring deposition. Moreover,

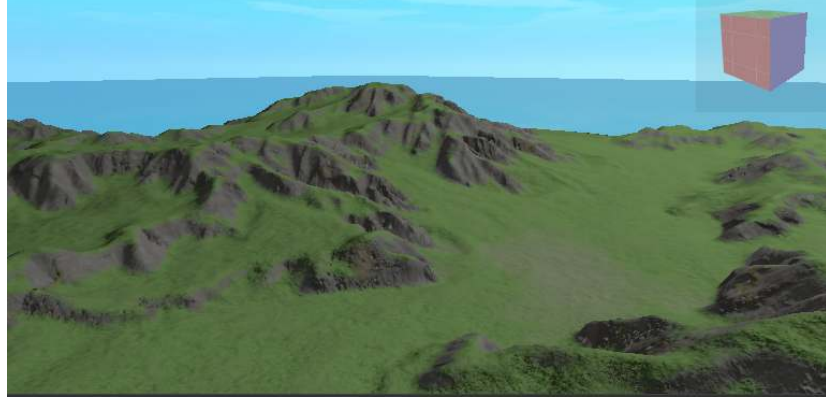


Figure 27: The terrain from Figure 26 after the application of hydraulic erosion. Simulation parameters: time step  $dt = 1.2$ , evaporation rate = 0.001, deposition rate  $k_c = 0.1$ , friction coefficient = 0.05, and particle density  $k = 1$ .

$c_{eq}$  is made proportional to the droplet's volume, velocity, and the elevation difference between the previous and new positions, as shown in Equation 17.

$$dc = dt \cdot k_C \cdot (c_{eq} - c) \quad (16)$$

$$c_{eq} = \max(0, \text{drop}_{volume} \cdot \text{drop}_{speed} \cdot (\text{height}_{old} - \text{height}_{new})) \quad (17)$$

If the equilibrium concentration is greater than the droplet's current sediment load, sediment is eroded from the terrain and absorbed. Conversely, if it is lower, sediment is deposited onto the terrain. The change in the heightmap is further scaled by the droplet's volume, as the terrain modification should be proportional to the mass being transferred rather than merely the concentration (see Equation 18). Additionally, at each simulation step, a portion of the droplet's volume is reduced due to evaporation. The rate of this reduction is governed by a user-defined evaporation rate, as shown in Equation 19.

$$\text{height}_{old} = \text{height}_{old} - \text{drop}_{volume} \cdot dc \quad (18)$$

$$\text{drop}_{volume} = \text{drop}_{volume} \cdot (1 - dt \cdot \text{evaporationRate}) \quad (19)$$

The effects of the erosion process are illustrated in Figure 27, which depicts the same landscape as in Figure 26 after applying erosion. In this example, five layers of erosion were simulated, with each droplet undergoing 500 simulation steps. The resulting terrain exhibits a noticeably more natural appearance, as mountain ridges are better defined and valleys have formed in lower elevation areas, contributing to a more realistic landscape morphology.

## 6 Implementation

This thesis was implemented within the *pyGANDALF* framework, which is described in Section 1.3. Two new systems were introduced, and the OpenGL rendering system was extended to support water rendering. The terrain generation system is responsible for creating the terrain grid and dispatching the compute shader that generates the heightmap. The erosion system, in turn, handles the dispatching of the compute shader that simulates hydraulic erosion.

### 6.1 Terrain Generation System

To generate the mesh of the terrain, a grid must first be defined. This grid dynamically follows the camera, and the camera’s coordinates are passed to the compute shader to be incorporated into the noise calculation. This approach creates the illusion of an infinite terrain, as new terrain is generated on-the-fly based on the camera’s position. The grid is composed of patches to support tessellation and needs to be constructed only once, as its structure remains static. However, if the scale parameter changes, the grid must be reconstructed to reflect the new scale accordingly as illustrated in Figure 28.

After the grid is initialized, the `vertices` and `tex_coords` arrays are assigned to the static mesh component. The `mesh.changed` flag is then set, prompting the updated vertices to be uploaded to the GPU. The generation process is triggered only when the `terrain.generate` flag is raised, which occurs either when the scale parameter is modified or when a custom heightmap is loaded (see Figure 29).

The second task of the system is to configure all the necessary uniforms for the compute shader responsible for heightmap generation. The shader is dispatched with one thread per pixel, where each thread computes the height value as detailed in Section 3.1. This is done using the unique coordinates `gl_GlobalInvocationID.xy`, which are offset by the camera coordinates to ensure continuity with camera movement. The computed height is then written to an output image for subsequent use.

### 6.2 Erosion System

The erosion system is responsible for dispatching the compute shader that performs the erosion simulation. One compute thread is launched per droplet, and every alternate frame simulates a single step of the droplet’s movement (see Figure 30). This incremental update approach is adopted to visually illustrate the gradual erosion process of the terrain. After each step, the state of every droplet is stored across two images. The first image encodes the droplet’s position using

```

53     if terrain.generate:
54         width, height = (terrain.scale, terrain.scale)
55
56         for i in range(terrain.patch_resolution):
57             for j in range(terrain.patch_resolution):
58                 vertex = []
59                 vertex.append((-width / 2.0 + width * i / float(terrain.patch_resolution))) # v.x
60                 vertex.append(0.0) # v.y
61                 vertex.append((-height / 2.0 + height * j / float(terrain.patch_resolution)) # v.z
62                 vertices.append(vertex)
63
64                 tex_coord = []
65                 tex_coord.append(i / float(terrain.patch_resolution)) # u
66                 tex_coord.append(j / float(terrain.patch_resolution)) # v
67                 tex_coords.append(tex_coord)
68
69                 vertex = []
70                 vertex.append((-width / 2.0 + width * (i + 1) / float(terrain.patch_resolution)) # v.x
71                 vertex.append(0.0) # v.y
72                 vertex.append((-height / 2.0 + height * j / float(terrain.patch_resolution)) # v.z
73                 vertices.append(vertex)
74
75                 tex_coord = []
76                 tex_coord.append((i + 1) / float(terrain.patch_resolution)) # u
77                 tex_coord.append(j / float(terrain.patch_resolution)) # v
78                 tex_coords.append(tex_coord)
79
80                 vertex = []
81                 vertex.append((-width / 2.0 + width * i / float(terrain.patch_resolution)) # v.x
82                 vertex.append(0.0) # v.y
83                 vertex.append((-height / 2.0 + height * (j + 1) / float(terrain.patch_resolution)) # v.z
84                 vertices.append(vertex)
85
86                 tex_coord = []
87                 tex_coord.append(i / float(terrain.patch_resolution)) # u
88                 tex_coord.append((j + 1) / float(terrain.patch_resolution)) # v
89                 tex_coords.append(tex_coord)
90
91                 vertex = []
92                 vertex.append((-width / 2.0 + width * (i + 1) / float(terrain.patch_resolution)) # v.x
93                 vertex.append(0.0) # v.y
94                 vertex.append((-height / 2.0 + height * (j + 1) / float(terrain.patch_resolution)) # v.z
95                 vertices.append(vertex)
96
97                 tex_coord = []
98                 tex_coord.append((i + 1) / float(terrain.patch_resolution)) # u
99                 tex_coord.append((j + 1) / float(terrain.patch_resolution)) # v
100                 tex_coords.append(tex_coord)
101
102     terrain.vertices = np.asarray(vertices, np.float32)
103     tex_coords = np.asarray(tex_coords, np.float32)
104
105     mesh.attributes = [terrain.vertices, tex_coords]
106     mesh.changed = True
107     terrain.generate = False
108

```

Figure 28: Code snippet from the `on_update_entity` function, responsible for constructing the grid as described in Section 3.2.

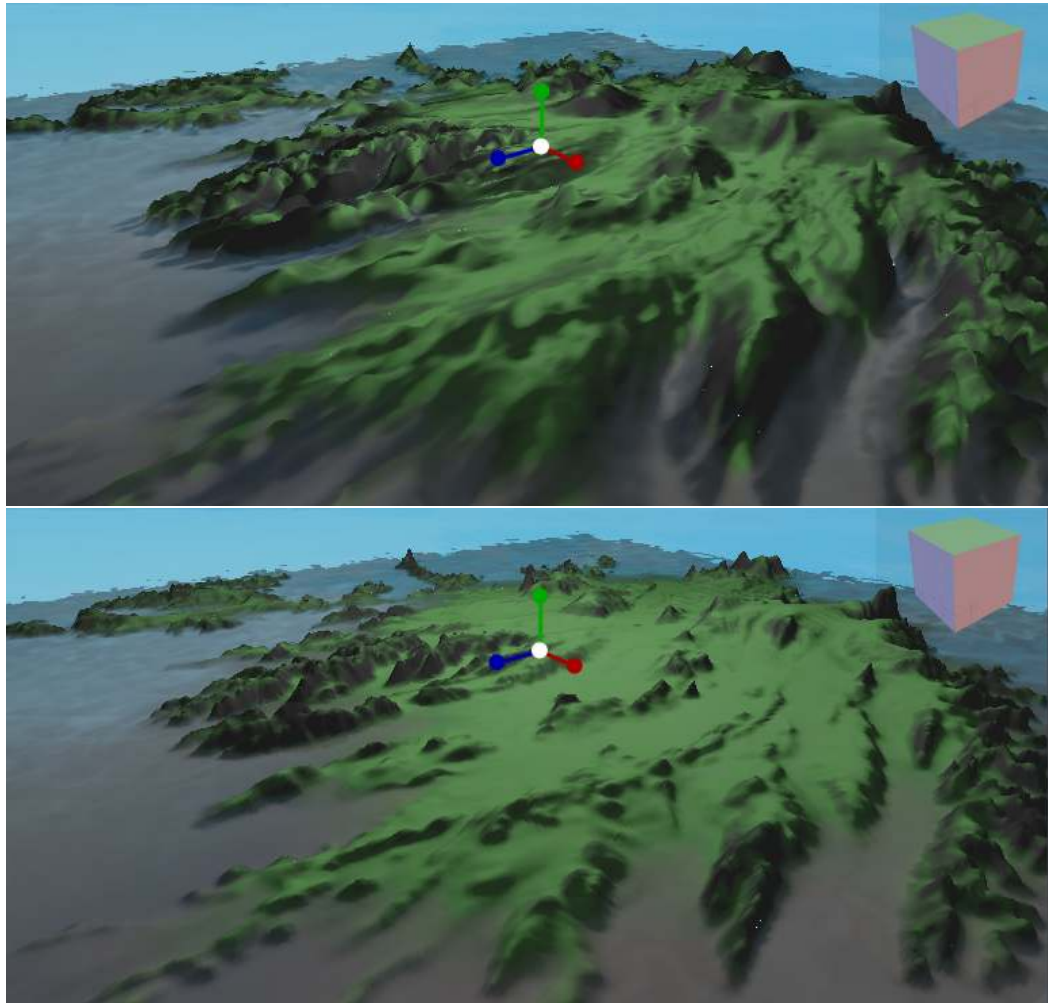


Figure 29: Terrain generated from a heightmap of Iceland (top), and the result after applying three successive layers of hydraulic erosion (bottom).

```

32 def on_update_entity(self, ts: float, entity: Entity, components: Component | tuple[Component]):
33     erosion: ErosionComponent = components[0]
34     terrain: TerrainComponent = components[1]
35
36     if erosion.enabled:
37         if erosion.counter % 2 == 0:
38             gl.glBindImageTexture(0, erosion.heightmapId, 0, gl.GL_FALSE, 0, gl.GL_READ_WRITE, gl.GL_RGBA32F)
39             gl.glBindImageTexture(1, erosion.dropsPosSpeedId, 0, gl.GL_FALSE, 0, gl.GL_READ_WRITE, gl.GL_RGBA32F)
40             gl.glBindImageTexture(2, erosion.dropsVolSedId, 0, gl.GL_FALSE, 0, gl.GL_READ_WRITE, gl.GL_RGBA32F)
41             gl.glBindImageTexture(3, erosion.normalsId, 0, gl.GL_FALSE, 0, gl.GL_READ_WRITE, gl.GL_RGBA32F)
42
43             gl.glUseProgram(erosion.erosionId)
44             location = gl.glGetUniformLocation(erosion.erosionId, 'started')
45             gl.glUniform1i(location, erosion.started)
46             location = gl.glGetUniformLocation(erosion.erosionId, 'elevationScale')
47             gl.glUniform1i(location, terrain.elevationScale)
48
49             gl.glDispatchCompute(erosion.width, erosion.height, 1)
50             gl.glMemoryBarrier(gl.GL_SHADER_IMAGE_ACCESS_BARRIER_BIT)
51             gl.glUseProgram(0)
52
53             erosion.counter += 1
54             erosion.started = 1
55             if erosion.counter == 1000:
56                 erosion.enabled = False
57
58         if erosion.save:
59             counter = 0
60             filename = "heightmap" + str(counter) + ".png"
61             while(path.isfile(path.abspath(filename))):
62                 counter += 1
63                 filename = "heightmap" + str(counter) + ".png"
64             self.export_texture(filename)
65             erosion.save = False

```

Figure 30: The `on_update_entity` function in erosion system.

the red and green channels to store the  $x$  and  $z$  coordinates (as the  $y$  component is implicitly given by the heightmap), while the blue and alpha channels store the droplet’s velocity, represented as a two-dimensional vector. The second image records the droplet’s physical state, with the red channel capturing the current volume and the green channel storing the amount of sediment it carries. In subsequent steps, these images are sampled so that the simulation continues from the previously computed state.

A simulation step counter is incremented at each frame to monitor the progression of the erosion process. The simulation concludes when the counter reaches 1000, indicating that each droplet has completed 500 movement steps across the terrain, as the simulation advances one step every two frames. Additionally, this system is responsible for extracting the resulting heightmap from the GPU and saving it to disk. This is achieved through the `export_texture` function, as illustrated in Figure 31. The heightmap is stored in the root directory of the *pyGANDALF* framework in grayscale PNG format.

### 6.3 Water Rendering

The approach used for rendering water in this system is based on the method demonstrated by ThinMatrix [23]. The existing OpenGL rendering system was extended with two additional rendering passes, each producing a full-scene output into a dedicated framebuffer. The first pass renders the scene with the camera mirrored along the water plane, while discarding all fragments above this plane.

```

64 def export_texture(self, filename, textureName: str = "heightmap"):
65     gl.glBindTexture(gl.GL_TEXTURE_2D, OpenGLTextureLib().get_id(textureName))
66     heightmap = gl.glGetTexImage(gl.GL_TEXTURE_2D, 0, gl.GL_RGBA, gl.GL_FLOAT)
67
68     image = Image.new('RGB', (heightmap.shape[0], heightmap.shape[1]), 0)
69     draw = ImageDraw.ImageDraw(image)
70     for z in range(heightmap.shape[0]):
71         for x in range(heightmap.shape[1]):
72             if math.isnan(heightmap[z][x][1]):
73                 heightmap[z][x][1] = 0.0
74             draw.point((z, x), (int(heightmap[z][x][1] * 255), int(heightmap[z][x][1] * 255), int(heightmap[z][x][1] * 255)))
75     image.save(filename)
76     print(filename, "saved")
77     return image.width, image.height

```

Figure 31: The `export_texture` function used to extract and save the computed heightmap from the GPU to disk.

This generates the reflection texture, capturing the reflections visible on the water surface. The second pass renders the scene from the normal camera position, discarding all fragments below the water plane. The result is a depth texture, which is subsequently used to simulate refraction effects (see Figure 32).

To render the water surface, a flat grid is generated at the same height as the water plane defined during the previous rendering passes. This grid is shaded using a separate fragment shader, distinct from the one used for the terrain. The shader utilizes the previously generated reflection and refraction textures and blends them using linear interpolation. To simulate the Fresnel effect, the interpolation factor is calculated by taking the square root of the dot product between the view direction vector and the surface normal of the water. As a result, when the camera looks directly down at the water surface, minimal reflection is observed, whereas reflections become more prominent at lower viewing angles.

To sample the reflection and refraction textures correctly, we must operate in normalized device coordinates (NDC). This is achieved by multiplying the vertex position of the water surface with the model-view-projection (MVP) matrix in the vertex shader and passing the resulting vector to the fragment shader. The NDC coordinates are then obtained by dividing the  $xy$  components of this vector by its  $w$  component. This produces values in the range  $[-1, 1]$ , which we convert to the  $[0, 1]$  range by dividing by 2 and adding 0.5. Note that coordinates of reflection are mirrored in  $y$  coordinate because the reflection needs to be sampled upside down. To compute the depth of the water, we must sample the depth map generated during the refraction rendering pass. This depth map stores the scene depth from the perspective of the camera. The depth of the current fragment, corresponding to the water surface, can be accessed through `gl_FragCoord.z`. To determine the relative depth of the water, we subtract the sampled refraction depth from the current fragment's depth (see Figure 34). Since OpenGL uses a non-linear depth buffer that provides greater precision for objects closer to the camera, we convert the depth values into linear distances prior to performing this subtraction. The relevant computations are illustrated in Figure 33.

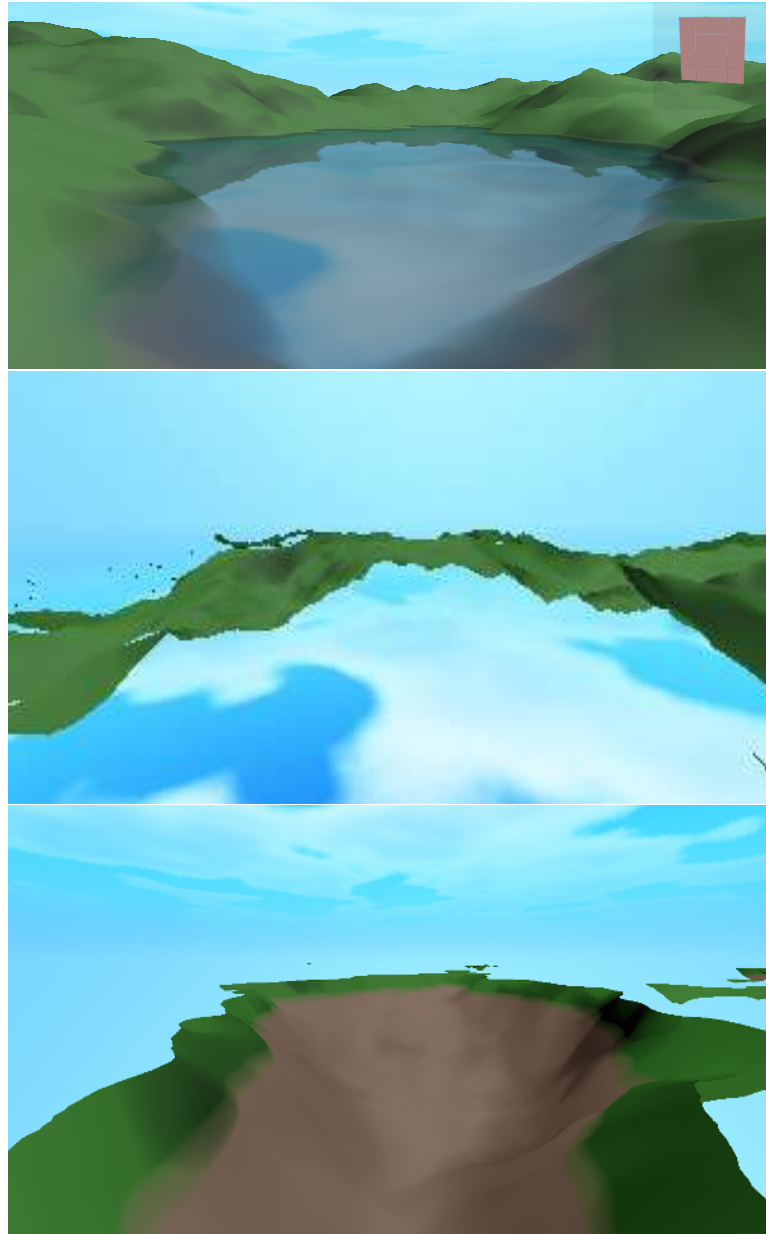


Figure 32: The rendered water surface (top), its corresponding reflection texture (middle), and the depth texture used for refraction (bottom). Note that the reflection texture is shown inverted vertically.

```

31 void main() {
32     vec2 normalizedDeviceCoords = ((clipSpace.xy/clipSpace.w) / 2.0) + 0.5;
33     vec2 refractionCoords = vec2(normalizedDeviceCoords.x, normalizedDeviceCoords.y);
34     vec2 reflectionCoords = vec2(normalizedDeviceCoords.x, -normalizedDeviceCoords.y);
35
36     float nearPlane = 0.1;
37     float farPlane = 2000.0;
38     float depth = texture(depthMap, refractionCoords).r;
39     float floorDistance = 2.0 * nearPlane * farPlane / (farPlane + nearPlane - (2.0 * depth - 1.0) * (farPlane - nearPlane));
40
41     depth = gl_FragCoord.z;
42     float waterDistance = 2.0 * nearPlane * farPlane / (farPlane + nearPlane - (2.0 * depth - 1.0) * (farPlane - nearPlane));
43     float waterDepth = floorDistance - waterDistance;

```

Figure 33: Code snippet from the water fragment shader demonstrating the calculation of normalized device coordinates and the water depth.

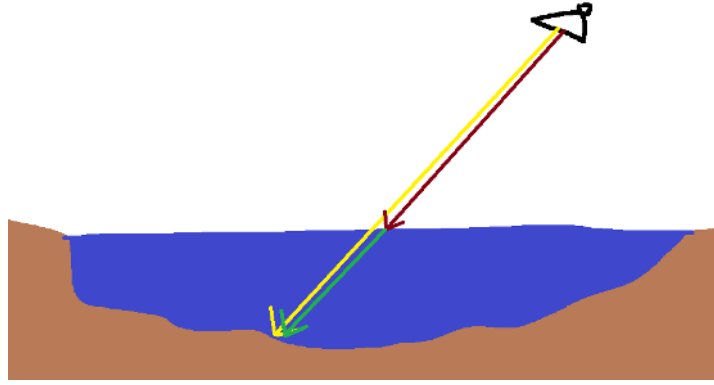


Figure 34: The red arrow represents the depth of the current water fragment (`gl_FragCoord.z`), the yellow arrow indicates the depth sampled from the refraction depth texture, and the green arrow shows the resulting water depth obtained by subtracting the two.

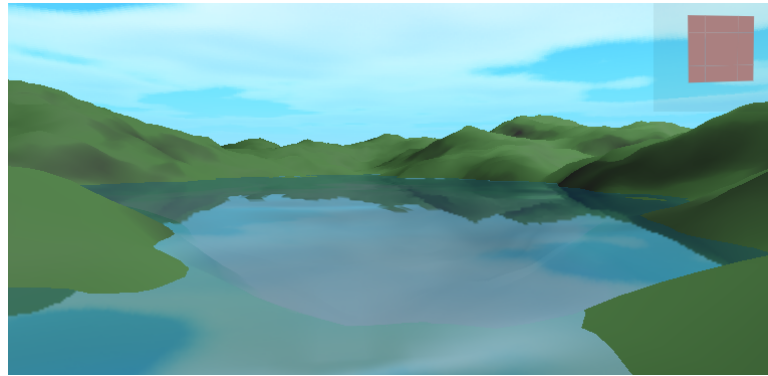


Figure 35: The landscape depicted in Figure 32 rendered without soft edge blending, resulting in visibly harsh transitions between water and terrain.

To achieve smoother transitions between water and terrain, the calculated water depth is used to perform alpha blending. Specifically, the alpha value of the water fragment's color is set to the water depth divided by a constant, resulting in lower alpha values (and therefore more blending) in shallower regions. This technique softens the edges of the water surface. To enable this behavior in OpenGL, alpha blending must be activated using the `GL_BLEND` flag along with the blend function (`GL_SRC_ALPHA`, `GL_ONE_MINUS_SRC_ALPHA`). Without this approach, sharp transitions at the water boundaries are visible, as shown in Figure 35.

To simulate the effect of dynamic, moving water, the reflection and refraction texture coordinates are perturbed using a displacement vector. This vector is obtained by sampling a DuDv (distortion) map, as illustrated in Figure 36, where the red and green channels are used to generate the distortion offset. The sampled values are transformed into the  $[-1, 1]$  range and added to the texture coordinates, which are then clamped to prevent invalid (i.e., negative) texture accesses. A

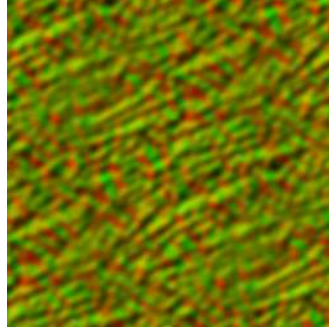


Figure 36: A dudv map. We sample this texture to offset the coordinates of the reflection and refraction textures.

```

46   vec2 distortedTexCoords = texture(dudvMap, vec2(texCoord.x + offset, texCoord.y)).rg * 0.1;
47   distortedTexCoords = texCoord + vec2(distortedTexCoords.x, distortedTexCoords.y + offset);
48
49   vec2 totalDistortion = (texture(dudvMap, distortedTexCoords).rg * 2.0 - 1.0) * waveStrength * smoothFactor;
50
51   refractionCoords += totalDistortion;
52   refractionCoords = clamp(refractionCoords, 0.001, 0.999);
53
54   reflectionCoords += totalDistortion;
55   reflectionCoords.x = clamp(reflectionCoords.x, 0.001, 0.999);
56   reflectionCoords.y = clamp(reflectionCoords.y, -0.999, -0.001);
57
58   vec4 refractionColor = texture(refraction, refractionCoords);
59   vec4 reflectionColor = texture(reflection, reflectionCoords);

```

Figure 37: Shader code responsible for applying dynamic distortion to the water surface using a DuDv map.

parameter, *waveStrength*, is introduced to control the intensity of the distortion. It is important to note that the y-coordinate of the reflection texture coordinates is clamped between  $-1$  and  $0$  due to the reflection inversion.

To animate the water surface over time, a uniform variable *offset* is introduced and incremented by a constant value each frame, wrapping around when it exceeds a value of 1 to maintain periodicity. This offset is initially applied to the *x*-component of the texture coordinates used to sample the DuDv map. The resulting distorted vector is then used to resample the DuDv map, this time applying the *offset* to the *y*-component of the coordinates (see Figure 37). By applying displacement in both horizontal and vertical directions, the distortion effect becomes more natural and fluid, enhancing the realism of the animated water surface.

## 6.4 Performance

The program was executed on a system equipped with an NVIDIA RTX 3050 GPU and an Intel Core i5-11400 CPU. Table 1 presents performance metrics for the terrain generation system. Notably, the terrain resolution does not exhibit a linear impact on performance, as heightmap generation is executed in parallel on the GPU. While the number of patches influences the time required to generate the terrain grid, this operation occurs only when the scale parameter changes and

therefore has a negligible impact on overall runtime performance.

Table 1: Performance of terrain generation system

| <b>Terrain resolution</b> | <b># of patches</b> | <b>Time (ms)</b> |
|---------------------------|---------------------|------------------|
| 512x512                   | 8                   | 5.6              |
| 256x256                   | 8                   | 3.4              |

As described in Section 6.2, the erosion system simulates one droplet per terrain grid point using compute shaders. Each droplet performs a single movement every other frame, with its state preserved and resumed in subsequent frames. This simulation spans 1000 frames, resulting in a total of 500 steps per droplet—sufficient to produce visually realistic hydraulic erosion. Increasing the number of simulation steps per compute shader dispatch introduces additional dependencies and intensifies image load/store operations, leading to significant performance degradation. Table 2 illustrates this effect, showing the number of frames (i.e., compute shader dispatches) and the corresponding execution time per frame.

Table 2: Performance of erosion system

| <b>Droplets</b> | <b># of GPU steps</b> | <b># of frames</b> | <b>Total steps</b> | <b>Time(ms)</b> |
|-----------------|-----------------------|--------------------|--------------------|-----------------|
| 512x512         | 1                     | 500                | 500                | 2.5             |
| 512x512         | 25                    | 20                 | 500                | 27              |
| 512x512         | 50                    | 10                 | 500                | 48              |
| 512x512         | 500                   | 1                  | 500                | 447             |

The overall performance of the erosion simulation averages approximately 500 ms, calculated as the product of the number of frames and the execution time per compute shader dispatch. As shown in Table 2, the last configuration yields the best total performance at 447 ms, while the first is the slowest at approximately 1250 ms ( $500 \cdot 2.6ms$ ). However, the final case introduces a noticeable delay, causing the application to pause for nearly half a second during erosion. In contrast, the first configuration maintains interactive responsiveness by minimizing per-frame execution time. Nevertheless, the total erosion duration still prevents real-time terrain generation and erosion from occurring within the same frame. To address this, the system disables infinite terrain generation while erosion is being computed.

## 7 Conclusion

This thesis presents a real-time system, implemented within the pyGANDALF framework, for procedurally generating infinite terrain and simulating hydraulic erosion interactively. The terrain is dynamically shaded using a physically based rendering (PBR) pipeline that blends three distinct textures based on elevation and an additional texture for steep slopes. The system provides flexibility by allowing users to configure various parameters, including terrain scale, resolution, noise functions, and shading properties. The framework achieves high performance, maintaining frame rates up to 144 FPS, and effectively visualizes terrain evolution due to erosion in real time. This makes it suitable for applications in interactive environments, such as games or simulations.

For future work, realism can be further enhanced through the integration of advanced rendering techniques such as precomputed radiance transfer, ambient occlusion, and displacement mapping. Additionally, the simulation could be extended by incorporating other erosion models, such as thermal erosion, to capture a wider range of natural terrain transformations.

## References

- [1] Hans Theobald Beye. Implementation of a method for hydraulic erosion. <http://www.firespark.de/resources/downloads/implementation%20of%20a%20methode%20for%20hydraulic%20erosion.pdf>, 2015. [Online; accessed October-2024].
- [2] Norishige Chiba, Kazunobu Muraoka, and Kunihiro Fujita. An erosion model based on velocity fields for the visual simulation of mountain scenery. *Comput. Animat. Virtual Worlds*, 9:185–194, 1998.
- [3] Alain Fournier, Don Fussell, and Loren Carpenter. Computer rendering of stochastic models. *Commun. ACM*, 25(6):371–384, June 1982.
- [4] Alain Fournier and William T. Reeves. A simple model of ocean waves. In *Proceedings of the 13th Annual Conference on Computer Graphics and Interactive Techniques*, SIGGRAPH '86, page 75–84, New York, NY, USA, 1986. Association for Computing Machinery.
- [5] Hello Games. No man's sky. <https://www.nomanssky.com/>, 2016. [Online; accessed March-2025].
- [6] Ryan Geiss. Chapter 1. Generating Complex Procedural Terrains Using the GPU. <https://developer.nvidia.com/gpugems/gpugems3/part-i-geometry/chapter-1-generating-complex-procedural-terrains-using-gpu>, 2007. [Online; accessed October-2024].
- [7] R. Steven Glanville. Texture bombing. In Randima Fernando, editor, *GPU Gems*, chapter 20. Addison-Wesley, 2004.
- [8] Jean-David Génevaux, Éric Galin, Éric Guérin, Adrien Peytavie, and Bedřich Beneš. Terrain generation using procedural models based on hydrology. *ACM Transactions on Graphics*, 32(4):143:1–143:13, 2013.
- [9] William E. Lorensen and Harvey E. Cline. Marching cubes: A high resolution 3d surface construction algorithm. In *Proceedings of the 14th Annual Conference on Computer Graphics and Interactive Techniques*, SIGGRAPH '87, page 163–169, New York, NY, USA, 1987. Association for Computing Machinery.

- [10] Benoît B. Mandelbrot. *The Fractal Geometry of Nature*. W. H. Freeman and Company, San Francisco, 1982.
- [11] Nicholas McDonald. Simple particle-based hydraulic erosion, 2020. [Online; accessed December-2024].
- [12] Xing Mei, Philippe Decaudin, and Bao-Gang Hu. Fast hydraulic erosion simulation and visualization on gpu. In *PG '07 - 15th Pacific Conference on Computer Graphics and Applications*, pages 47–56. IEEE, 2007.
- [13] Gavin S P Miller. The definition and rendering of terrain maps. *SIGGRAPH Comput. Graph.*, 20(4):39–48, August 1986.
- [14] Andrey Mishkinis. Advanced terrain texture splatting, 2013. [Online; accessed December-2024].
- [15] F. K. Musgrave, C. E. Kolb, and R. S. Mace. The synthesis and rendering of eroded fractal terrains. In *Proceedings of the 16th Annual Conference on Computer Graphics and Interactive Techniques*, SIGGRAPH '89, page 41–50, New York, NY, USA, 1989. Association for Computing Machinery.
- [16] World of Level Design. UE4: Secrets to Generating Real-World Heightmaps from Terrain.Party for UE4. <https://www.worldofleveldesign.com/categories/ue4/landscape-heightmap-terrain-party.php>, 2020. [Online; accessed March-2025].
- [17] Dr. Jeffrey Paone. Learn OpenGL Tessellation Chapter I: Rendering Terrain using Height Maps. <https://learnopengl.com/Guest-Articles/2021/Tessellation/Height-map>, 2021. [Online; accessed November-2024].
- [18] Dr. Jeffrey Paone. Learn OpenGL Tessellation Chapter II: Rendering Terrain using Tessellation Shaders & Dynamic Levels of Detail. <https://learnopengl.com/Guest-Articles/2021/Tessellation/Height-map>, 2021. [Online; accessed November-2024].
- [19] Ken Perlin. An image synthesizer. In *Proceedings of the 12th Annual Conference on Computer Graphics and Interactive Techniques*, SIGGRAPH '85, page 287–296, New York, NY, USA, 1985. Association for Computing Machinery.
- [20] John Petropoulos, Manos Kamarianakis, Antonis Protopsaltis, and George Papagiannakis. pygandalf – an open-source, geometric, animation, directed, algorithmic, learning framework for computer graphics, 2024.

- [21] Planetside Software. Terragen. <https://planetside.co.uk/>, 2005. [Online; accessed March-2025].
- [22] Mojang Studios. Minecraft. <https://www.minecraft.net/en-us>, 2011. [Online; accessed March-2025].
- [23] ThinMatrix. Opengl water tutorial, 2015. [Online; accessed December-2024].
- [24] Jose Villegas. Heightmap Generator. <https://github.com/jose-villegas/HeightmapGenerator>, 2015. [Online; accessed March-2025].
- [25] Richard F. Voss. Random fractal forgeries. In *Fundamental Algorithms for Computer Graphics*, pages 805–835. Springer, 1983.
- [26] Georgios Voulgaris, Ioannis Mademlis, and Ioannis Pitas. Procedural terrain generation using generative adversarial networks. In *2021 29th European Signal Processing Conference (EUSIPCO)*, pages 686–690, 2021.
- [27] Joey De Vries. Learn OpenGL PBR Theory. <https://learnopengl.com/PBR/Theory>, 2016. [Online; accessed November-2024].
- [28] Polycount Wiki. Polycount Wiki Splat. <http://wiki.polycount.com/wiki/Splat>, 2016. [Online; accessed April-2025].
- [29] Wikipedia. Perlin noise. [https://en.wikipedia.org/wiki/Perlin\\_noise](https://en.wikipedia.org/wiki/Perlin_noise), 2004. [Online; accessed October-2024].
- [30] Wikipedia. Diamond-square algorithm. [https://en.wikipedia.org/wiki/Diamond-square\\_algorithm](https://en.wikipedia.org/wiki/Diamond-square_algorithm), 2005. [Online; accessed March-2025].
- [31] Wikipedia. Marching cubes. [https://en.wikipedia.org/wiki/Marching\\_cubes](https://en.wikipedia.org/wiki/Marching_cubes), 2005. [Online; accessed October-2024].
- [32] Thomas A. Witten and Leonard M. Sander. Diffusion-limited aggregation, a kinetic critical phenomenon. *Physical Review Letters*, 47(19):1400–1403, 1981.
- [33] Procedural Worlds. Gaia. <https://www.procedural-worlds.com/products/indie/gaia/>, 2015. [Online; accessed March-2025].